

NAFLD Prediction with Logistic Regression

University of London

By **Mayra Torres** July-2023

Table of Contents

Introduction

1. Domain, Area of Investigation
2. Dataset

Implementation

1. Data Preprocessing
2. Statistical Analysis: Distribution
3. Feature Engineering First Iteration
4. Statistical Analysis: Central Tendency and Variability
5. Data Representation and Visualization
6. Feature Engineering Second Iteration
7. Machine Learning Model

Model Evaluation

1. Accuracy Scores
2. Confusion Matrix
3. Classification Report
4. Balanced Accuracy
5. ROC Curve
6. Precision-Recall Curve
7. Feature Importance

Conclusion

1. Project Reflection

References

Introduction

[Table of Contents Link](#)

Domain, Area of Investigation

Aims and Objectives

The aim of this report is to present a data analysis on the characteristics and possible causes of a population's sample of non-alcoholic fatty liver disease NAFLD and provide information that predicts the presence of disease on the subject.

The objectives of the report are:

1. Describe the area and findings of the research and the reasons and motivations behind it.
2. Find a suitable data set and describe its components and characteristics.
3. Preprocess the data by organizing and cleaning the dataset.
4. Apply feature engineering to convert the raw data set into data features for analysis.
5. Apply data statistics techniques to the data features to provide a summary of the information.
6. Present a visualization through graphs of the findings.
7. Present a model validation to show the accuracy score.
8. Conclude and evaluate the model results.

Description

The study of the body and its different conditions is an extensive field of investigation. However, nowadays, it is easier to find data from different sources online. Data creates a big impact on the healthcare industry. Moreover, it facilitates the study and analysis of patterns that can help understand how the body works and how to improve the lives of the patients. Another benefit of studying medical data is to be able to have a health care system where processes are more efficient. Other benefits are "supporting clinical treatment decisions" (Joshi) and "improving speed and accuracy on identifying diseases" (Joshi).

One branch of medicine that can be benefited from further research is hepatology, "that studies the prevention, diagnosis, and management of diseases that affect the liver, gallbladder, biliary tree, and pancreas" (Robertson). For example, cirrhosis is the "11th leading cause of death" (Cheemerla) in the world. And the "The most common cause of prevalent disease is non-alcoholic fatty liver disease NAFLD with 59%" (Cheemerla). One of the leading causes of NAFLD is obesity and type 2 diabetes. "In 2016, more than 1.9 billion adults, 18 years and older, were overweight" (WHO.int) and more than half a million were obese. Furthermore, it is crucial to understand how the weight and BMI predicts liver diseases.

For that reason, this research report focuses on understanding how certain characteristics and conditions of the human body affect the diagnosis of non-alcoholic fatty liver disease. Another reason is that liver diseases are often preventable. Therefore, by expanding the study of the data available and the data that every day is gathered we can make a big impact on people's health.

Dataset

A publication on the American Association for the Study of Liver Diseases AASLD, “the leading organization of scientists and health care professionals committed to preventing and curing liver disease” (AASLDpubs), presented the findings of a 20 year study on non-alcoholic fatty liver disease NAFLD. NAFLD occurs when there is build up of fats in the liver in people that drink little to no alcohol.

The study was done in a “community cohort of all adults diagnosed with NAFLD in Olmsted County, Minnesota, between 1997 and 2014 using the Rochester Epidemiology Project database” (Allen et al.). All medical data collected from the medical providers in this community was linked through an ID number in a system.

The dataset from this study is named Non-alcohol fatty liver disease (NAFLD) found in Kaggle.com (Singh). The data file contains the baseline information about the characteristics of the subject.

The following are the variables of the dataset:

1. **Subject identifier (Unnamed):** an arbitrary number given to each subject across medical services. It doesn't add any value to the predictions.
2. **ID number for each sample (id):** identifier integer for each subject.
3. **Age (age):** the age of the subject at the time of the entry to the study. This can influence the metabolic process of the body.
4. **Male (male):** whether the subject is a male. If the person is a male it is represented with 1(true) otherwise with 0 (false).
5. **Weight (weight):** the weight of the person is represented in kilograms. It is shown that the weight of the person can have big effects on the overall health of the people.
6. **Height (height):** the height of the person represented in centimeters. This affects the BMI calculation in combination with the weight.
7. **BMI (bmi):** the Body Mass Index is a measurement of fat calculated using weight and height. It is a ratio that better describes the amount of fat in a person's body.
8. **Case.ID (case.id):** is a number id that represents the NAFLD cas. However, this column does not add value to the predictions.
9. **Futime (futime):** it is the time to death or to the last medical follow up.
10. **Status (status):** This column represents the status of the person at the last follow up. It is represented with 0 as alive and 1 as dead.

Implementation

[Table of Contents Link](#)

Data Preprocessing

Import Libraries

Python can be used with many libraries that add support to data wrangling. First, we need to import packages and libraries to access them in the Python notebook.

Numpy: adds support to the array handling and complex mathematical functions.

Pandas: designed for Python and supports data manipulation and data wrangling. It helps to handle data and time series and tables.

Matplotlib: is a broad library for plotting and visualizing data with high customization capabilities. It adds static and interactive visualization options.

Seaborn: is a library based on matplotlib with an object oriented design that makes seaborn more intuitive than matplotlib.

```
In [1]: # Import data management libraries and packages
import numpy as np
import pandas as pd
import statistics

# Import data visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
```

Import, Understand and Copy the Data

The input data is stored locally, and it can be accessed from the same file as this notebook: `"/nafld1.csv"`.

```
In [ ]: # Extract the data
data = pd.read_csv('data/nafld1.csv')
```

```
In [3]: # Count the number of features
numCol = len(data.columns)
numCol
```

Out[3]: 10

```
In [4]: # Print the data types for each column.
data.dtypes
```

```
Out[4]: Unnamed: 0      int64
id              int64
age             int64
male            int64
weight          float64
height          float64
bmi             float64
case.id         float64
fuptime         int64
status          int64
dtype: object
```

```
In [5]: # Shows the content of the dataset
data.head()
```

```
Out[5]:
```

	Unnamed: 0	id	age	male	weight	height	bmi	case.id	futime	status
0	3631	1	57	0	60.0	163.0	22.690939	10630.0	6261	0
1	8458	2	67	0	70.4	168.0	24.884028	14817.0	624	0
2	6298	3	53	1	105.8	186.0	30.453537	3.0	1783	0
3	15398	4	56	1	109.3	170.0	37.830100	6628.0	3143	0
4	13261	5	68	1	NaN	NaN	NaN	1871.0	1836	1

As we can see the dataset has 10 columns, but not all of them add value to the analysis or the value is minimum and we already have many features. Another characteristic is that some entities or items are missing some values.

A copy of the data is created before modifying the data, in order to preserve the original data.

```
In [6]: # Make a copy of the data
nafld_data = data.copy()
```

Remove Columns with Little or Non Value

When visualizing all the columns, I can see that there are columns that don't add value to the analysis. This is because the data that represents is to identify the sample but not to describe it. In the following steps, the columns that are not needed will be removed.

```
In [7]: # Gets the names of the columns or variables.
cols = nafld_data.columns
print(cols)
```

```
Index(['Unnamed: 0', 'id', 'age', 'male', 'weight', 'height', 'bmi', 'case.id',
      'futime', 'status'],
      dtype='object')
```

```
In [8]: print('Data description:')
nafld_data.describe().style.format("{:.2f}")
```

Data description:

Out[8]:

	Unnamed: 0	id	age	male	weight	height	bmi	case.id	1
count	17549.00	17549.00	17549.00	17549.00	12763.00	14381.00	12588.00	17518.00	17
mean	8780.11	8784.22	52.66	0.47	86.35	169.43	30.07	8840.92	2
std	5070.54	5070.97	14.72	0.50	22.24	10.14	7.09	5051.33	1
min	1.00	1.00	18.00	0.00	33.40	123.00	9.21	3.00	
25%	4389.00	4393.00	42.00	0.00	70.00	162.00	25.14	4598.00	1
50%	8778.00	8786.00	53.00	0.00	83.90	169.00	28.88	8781.00	2
75%	13171.00	13175.00	63.00	1.00	99.20	177.00	33.71	13249.00	3
max	17566.00	17566.00	98.00	1.00	181.70	215.00	84.40	17563.00	7



Based on the dataset description and the previous table, the columns that add **no** value to this analysis to be removed are:

1. Subject identifier (Unnamed)
2. ID number for each sample (id)
3. Case.ID (case.id)

Due to having too many features, for a have clear conclusion of the model, the columns that add **little** value to this analysis will be removed:

```
In [9]: # Drop or remove the columns
nafld = nafld_data.drop([cols[0], cols[1], cols[7], cols[8]], axis=1)
nafld
```

```
Out[9]:
```

	age	male	weight	height	bmi	status
0	57	0	60.0	163.0	22.690939	0
1	67	0	70.4	168.0	24.884028	0
2	53	1	105.8	186.0	30.453537	0
3	56	1	109.3	170.0	37.830100	0
4	68	1	NaN	NaN	NaN	1
...	...	...	...	...	...	...
17544	46	0	53.0	161.0	20.501023	0
17545	52	1	111.8	154.0	47.335905	0
17546	59	0	57.3	NaN	NaN	0
17547	61	0	NaN	NaN	NaN	1
17548	69	1	94.1	180.0	29.204650	0

17549 rows × 6 columns

Normalized Data

Normalize the data by organizing the data in a way that appears consistent across entities to eliminate redundancy. One method is to verify the data table is of First Normal Form 1NF. 1NF is set of rules to organize the data to meet the following rules (Tutorialspoint.com):

1. Every column should contain atomic values, values that can not be further divided.
2. No repeated groups or columns.

```
In [10]: # Function to find the maximum and minimum value in a cell in a given column
def NumberOfValues(col):
    cell_val = [len([i]) for i in nafld[col]]
    print ('    Max value: ', max(cell_val))
    print ('    Min value: ', min(cell_val))
```

```
In [11]: # Function to iterate the columns and call the NumberOfValues function
def MaxMinInColumn(cols):
    print('Maximum and Minimum value for each column: ')
    for col in cols:
        print(' ', col.upper())
        NumberOfValues(col)
```

```
In [12]: # Calls the previous function with the columns names as a list to find the min and
MaxMinInColumn(nafld.columns)
```

Maximum and Minimum value for each column:

AGE

Max value: 1

Min value: 1

MALE

Max value: 1

Min value: 1

WEIGHT

Max value: 1

Min value: 1

HEIGHT

Max value: 1

Min value: 1

BMI

Max value: 1

Min value: 1

STATUS

Max value: 1

Min value: 1

As we can see from the previous analysis, the table already contains one value for each cell. Therefore, we met the two rules for 1NF because each cell can not be further divided and we don't have repeated columns.

Statistical Analysis: Distribution

[Table of Contents Link](#)

Statistical Analysis is a method for extracting and interpreting tabular data.

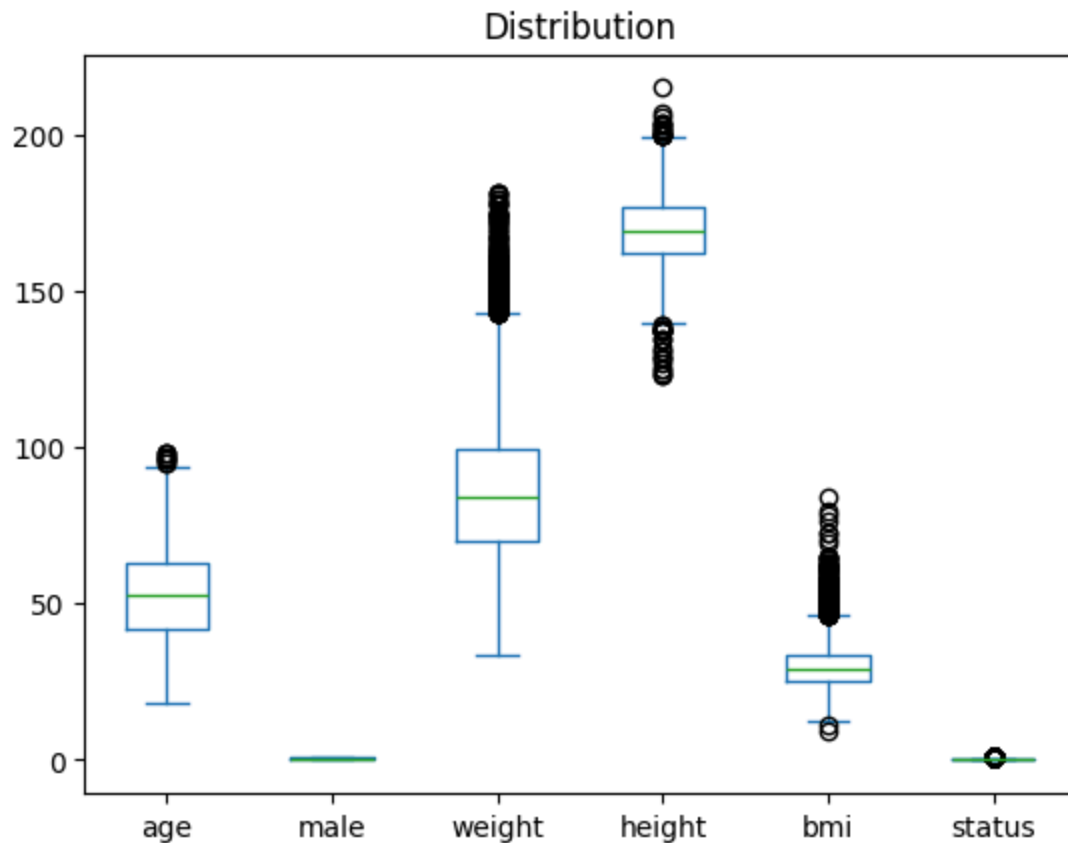
There are two measures for statistical analysis:

1. Distribution
2. Central Tendency
3. Variability

Distribution

It shows how the values are distributed. More over it is the probability of "all outcomes that a random variable takes" (GeeksforGeeks)

```
In [13]: # Distribution of the data
dis_plot = nafld.plot.box()
dis_plot.set_title('Distribution')
plt.show()
```



From the previous plot, we can say that the features age, weight, height and bmi have many outliers. This fact tells us that using the mean to fill NaN cells will yield a bias result. For this reason, the median will be used for imputed values because it is not easily affected by the outliers.

Feature Engineering First Iteration

[Table of Contents Link](#)

Feature engineering is a technique for selecting and wrangling raw data into features that are suitable for learning models. The process includes:

1. **Cleaning and inputting data** is cleaning and organizing the data including removing errors and empty spaces, or finding suitable data to replace the error or empty space.
2. **Data transformation** is the process of scaling and transforming variables into usable features.
3. **Feature extraction** is where new features are created using existing ones.
4. **Feature selection** is the process of choosing the features or variables that are relevant for the analysis.
5. **Feature iteration** is the step of reviewing, refining and improving the features.
6. **Feature split** is the process of dividing features into multiple variables.

Cleaning and Inputting Data

In this step, the NaN values will be analyzed and replaced for new usable values:

```
In [14]: # Finds the ratio of number of empty cells for each column
print('Number of entities per feature: ', len(nafld))
nan_num = nafld.isna().sum()
nan_ratio = nan_num / len(nafld)
pd.DataFrame((nan_num, nan_ratio), index=('NaN Count', 'NaN Ratio'))
```

Number of entities per feature: 17549

```
Out[14]:
```

	age	male	weight	height	bmi	status
NaN Count	0.0	0.0	4786.000000	3168.000000	4961.000000	0.0
NaN Ratio	0.0	0.0	0.272722	0.180523	0.282694	0.0

As we can see from the previous chart, the number of missing values is relatively high. However, all the missing values are in the columns weight, height and BMI. This gives the opportunity to see if one of the missing variables can be calculated if we still have two of the other variables.

Find the number of missing values for each sample to find the sample with only one missing variable

```
In [15]: # Count the number of empty cells for each row
nan_row = nafld.isna().sum(axis=1)
print("Empty cells per row:")
print(nan_row)
```

Empty cells per row:

```
0      0
1      0
2      0
3      0
4      3
```

..

```
17544   0
17545   0
17546   2
17547   3
17548   0
```

Length: 17549, dtype: int64

```
In [16]: # finds if there are rows with only one missing value.
print('Count of number of rows with a number of empty cells:')
nan_rows = pd.DataFrame(nan_row.value_counts())
nan_rows.columns = ['Number of rows']
nan_rows
```

Count of number of rows with a number of empty cells:

Out[16]: **Number of rows**

0	12588
3	2993
2	1968

As we can see, the missing values can not be calculated because there are no rows with one missing value. The rows are missing two or three values per sample.

In the following step, the missing values will be filled with median values

Fill with median value

```
In [17]: # Create a copy of the data to be able to fill the NaN cells
nafld_median = nafld.copy()
```

```
In [18]: # Fill each column NaN cells with median value
for col in nafld_median.columns:
    nafld_median[col] = nafld_median[col].fillna(nafld_median[col].median())

nafld_median.head()
```

```
Out[18]:
```

	age	male	weight	height	bmi	status
0	57	0	60.0	163.0	22.690939	0
1	67	0	70.4	168.0	24.884028	0
2	53	1	105.8	186.0	30.453537	0
3	56	1	109.3	170.0	37.830100	0
4	68	1	83.9	169.0	28.875884	1

Once the empty cells are filled with median data we can proceed to calculate the central tendency and variability of the data.

Statistical Analysis: Central Tendency and Variability

[Table of Contents Link](#)

Central Tendency

Central tendency is a way to describe a value that represents a data set. There are three methods that describe the measures of central tendency:

1. Mean
2. Median
3. Mode

In the following section I will calculate the measures of central tendency for the data set.

```
In [19]: # function for calculating the central tendency measurments
def CalculateCT(cols, func, ct_name, dictny):
    print(ct_name.upper())
    col_arr = []
    for col in cols:
        print(' ', col.upper(), ': ', func(nafl_d_median[col]))
        col_arr.append(func(nafl_d_median[col]))
    if ct_name == 'mean':
        dictny['mean'] = col_arr
    elif ct_name == 'median':
        dictny['median'] = col_arr
    else:
        dictny['mode'] = col_arr
```

```
In [20]: # Calculate the central tendency of the data
stats_cols = ['age', 'weight', 'height', 'bmi']
ct_dictny = {'mean': None, 'median': None, 'mode': None}
# Calculate the mean
print(CalculateCT(stats_cols, statistics.mean, 'mean', ct_dictny))
# Calculate the median
print(CalculateCT(stats_cols, statistics.median, 'median', ct_dictny))
# Calculate the mode
print(CalculateCT(stats_cols, statistics.mode, 'mode', ct_dictny))
```

MEAN

```
AGE : 52.65963872585333
WEIGHT : 85.68426690979543
HEIGHT : 169.35643056584422
BMI : 29.735202765286818
```

None

MEDIAN

```
AGE : 53
WEIGHT : 83.9
HEIGHT : 169.0
BMI : 28.8758842891644
```

None

MODE

```
AGE : 50
WEIGHT : 83.9
HEIGHT : 169.0
BMI : 28.8758842891644
```

None

Variability

Variability is how the data is spread across the ranges.

There are three methods for describing the dataset variability:

1. Range
2. Variance
3. Standard deviation

```
In [21]: # Function to calculate the range of a column
def CalcRange(col):
    return max(col) - min(col)
```

```
In [22]: # function for calculating the variability measurments
def CalculateVar(cols, func, var_name, dictny):
    print(var_name.upper())
    col_arr = []
    for col in cols:
        print(' ', col.upper(), ': ', func(nafld_median[col]))
        col_arr.append(func(nafld_median[col]))
    if var_name == 'range':
        dictny['range'] = col_arr
    elif var_name == 'variance':
        dictny['variance'] = col_arr
    else:
        dictny['sd'] = col_arr
```

```
In [23]: # Calculate the variability of the data
var_dictny = {'range': None, 'variance': None, 'sd': None}
# Calculate the range
print(CalculateVar(stats_cols, CalcRange, 'range', var_dictny))
# Calculate the variance
print(CalculateVar(stats_cols, statistics.variance, 'variance', var_dictny))
# Calculate the standard deviation
print(CalculateVar(stats_cols, statistics.stdev, 'standard deviation', var_dictny))
```

RANGE

```
AGE : 80
WEIGHT : 148.29999999999998
HEIGHT : 92.0
BMI : 75.18839345884729
```

None

VARIANCE

```
AGE : 216.75245168164588
WEIGHT : 360.8931243499997
HEIGHT : 84.30884014193325
BMI : 36.30141244405003
```

None

STANDARD DEVIATION

```
AGE : 14.72251512757402
WEIGHT : 18.99718727469937
HEIGHT : 9.181984542675577
BMI : 6.025065347699561
```

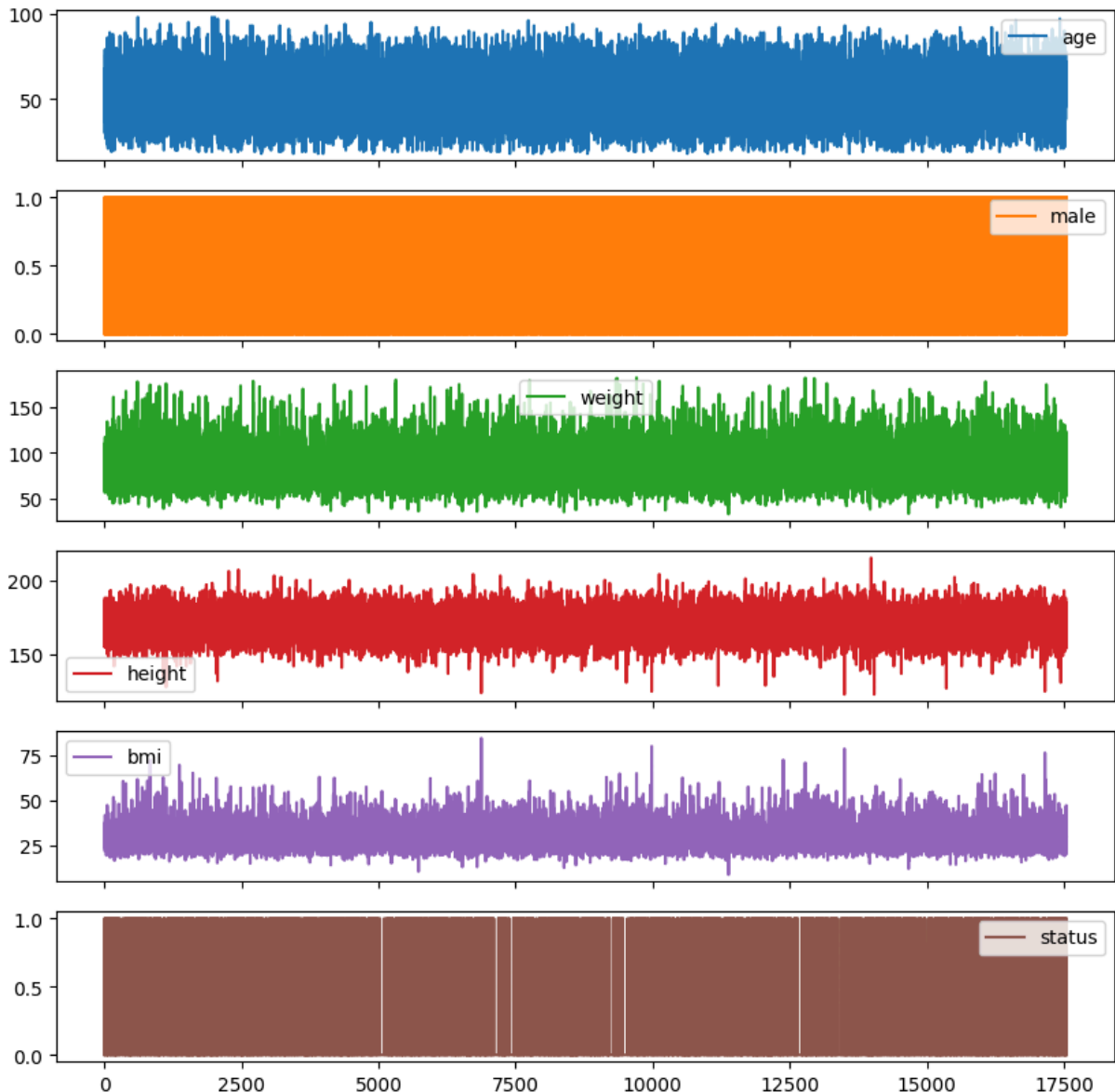
None

Data Representation and Visualization

[Table of Contents Link](#)

During data visualization, data is represented in chart tables that can show information in an organized manner. However, drawing conclusions is not that easy or clear. A better representation is in the form of graphs and visualizations. Visualization facilitates the display of data in a manner that the user can interpret the data into information and draw conclusions from it. For this we will analyze each feature on its own to understand the cause of each of the variability results.

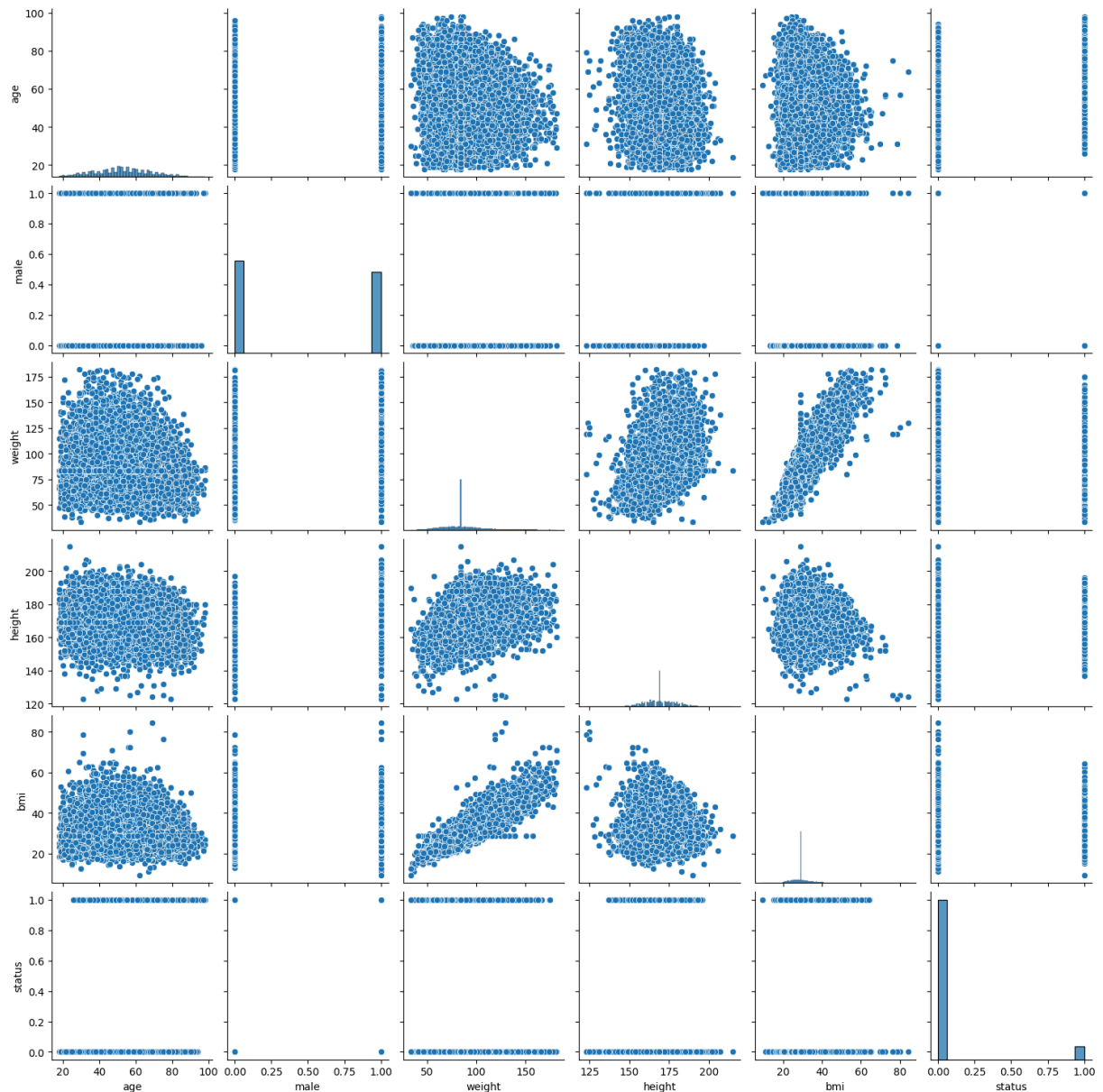
```
In [24]: # Shows all features in one set of plots
nafld_median.plot(subplots=True, layout=(6,1), figsize=(10,10))
plt.show()
```



The following plot shows the distribution and relationship of each variable. It gives us a good insight of which features have the potential to be studied.

```
In [25]: # Draw a plot of the relationship between variables
sns.pairplot(nafld_median)
```

Out[25]: <seaborn.axisgrid.PairGrid at 0x770aeeb51660>



One observation from this plot is that the weight and BMI are related and dependent from each other. Height also shows a relationship with the BMI however is not that dependent on each other.

Visualizing statistical information

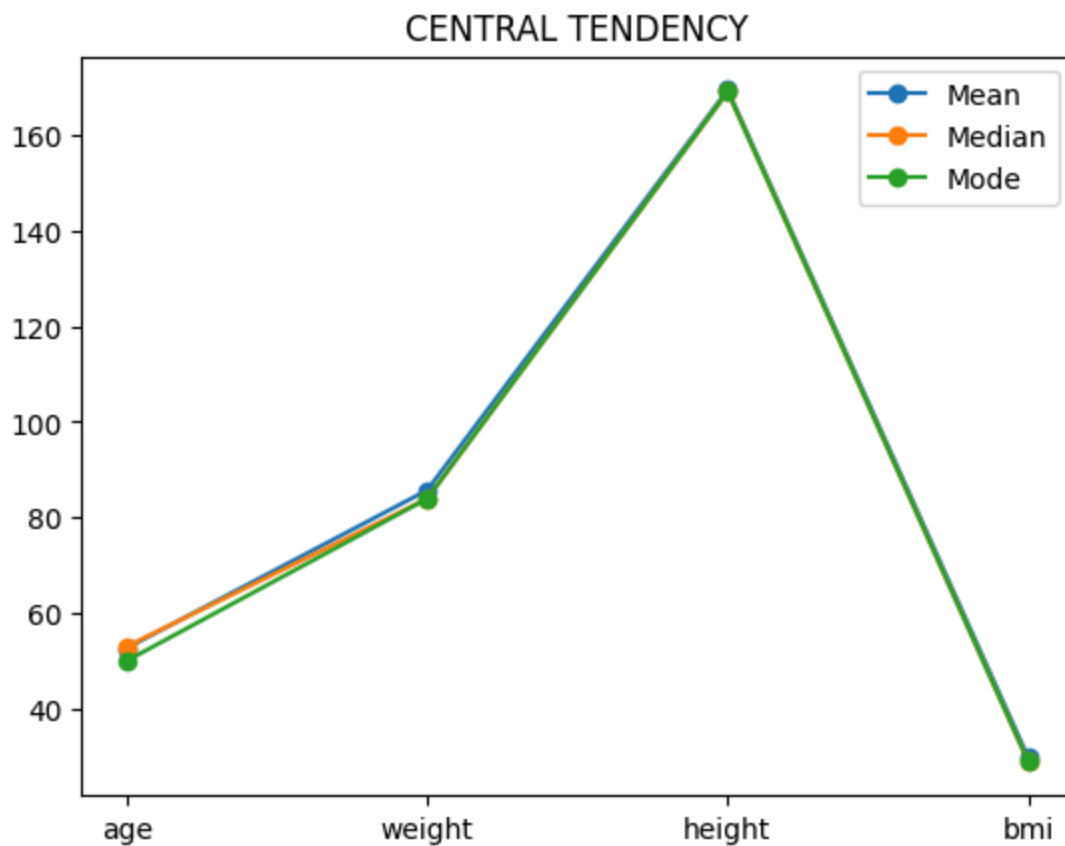
The following are the charts and graphs representing the values of central tendency and variability.

```
In [26]: # Using the dictionary from the central tendency calculations, creates a table
stats_ct_DF = pd.DataFrame(ct_dictny, index=stats_cols)
stats_ct_DF
```

Out[26]:

	mean	median	mode
age	52.659639	53.000000	50.000000
weight	85.684267	83.900000	83.900000
height	169.356431	169.000000	169.000000
bmi	29.735203	28.875884	28.875884

```
In [27]: plt.title("CENTRAL TENDENCY")
plt.plot(stats_ct_DF['mean'], label='Mean', marker = 'o')
plt.plot(stats_ct_DF['median'], label='Median', marker = 'o')
plt.plot(stats_ct_DF['mode'], label='Mode', marker = 'o')
plt.legend(loc='best')
plt.show()
```



From this chart we can see that the central tendency values are very similar, therefore most of the values gather around the same respective values for each subject's characteristics.

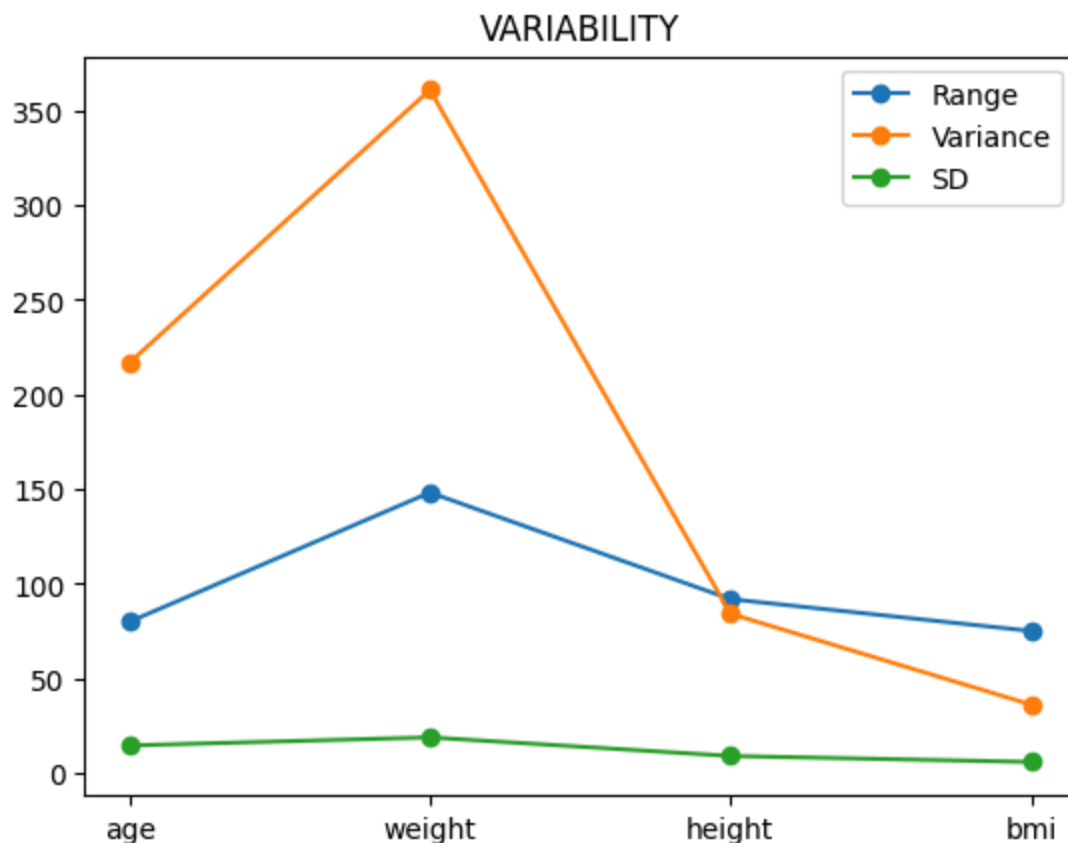
```
In [28]: # Using the dictionary from the variability calculations, creates a table
stats_var_DF = pd.DataFrame(var_dictny, index=stats_cols)
stats_var_DF
```

Out[28]:

	range	variance	sd
age	80.000000	216.752452	14.722515
weight	148.300000	360.893124	18.997187
height	92.000000	84.308840	9.181985
bmi	75.188393	36.301412	6.025065

```
In [29]: plt.title("VARIABILITY")
plt.plot(stats_var_DF['range'], label='Range', marker = 'o')
plt.plot(stats_var_DF['variance'], label='Variance', marker = 'o')
plt.plot(stats_var_DF['sd'], label='SD', marker = 'o')
plt.legend(loc='best')
```

Out[29]: <matplotlib.legend.Legend at 0x770aee520700>



From this chart, we can see that the variance values tell us more information about the data.

1. The variance shows us that the values are vastly dispersed from the mean for all the features, specially for the weight feature.
2. The range line shows that the values have an ample range for each feature, this gives us the opportunity to analyze most of the population's characteristics, however it shortens the precision of the observations.

3. The standard deviation, even though it is located at the bottom of the graph, is relatively high, meaning the data have many outliers that skew the results.

For this we will analyze each feature on its own to understand the cause of each of the variability results.

In the following sections, variables or features will be visualized on their own to understand the data distribution.

- **Age Visualisation**

```
In [30]: # Function to format the subplots
def FormatPlots(ax1, ax2, xs, ys, q1, q3, inter_qr, median, pctg):
    #Colors the quartiles on the probability density function graph
    ylims = ax1.get_ylim()
    ax2.fill_between(xs, 0, ys, where=(xs >= q1 - 1.5 * inter_qr) & (xs <= q3 + 1.5
    ax2.fill_between(xs, 0, ys, where=(xs >= q1) & (xs <= q3), color='white', alpha

    # Quartiles
    ax1.text(q1 - 2 * inter_qr, -0.8, "%.2f" % pctg['L.outlier'] + '%', ha='center'
    ax1.text(q1 - (inter_qr / 2), -0.8, "%.2f" % pctg['25'] + '%', ha='center', va=
    ax1.text(q1 + (inter_qr / 2), -0.8, "%.2f" % pctg['IQR'] + '%', ha='center', va
    ax1.text(q3 + (1.5 * inter_qr / 2), -0.8, "%.2f" % pctg['75'] + '%', ha='center
    ax1.text(q3 + 2 * inter_qr, -0.8, "%.2f" % pctg['R.outlier'] + '%', ha='center'

    ax1.text(q1, 0.6, 'Q1', ha='center', va='center', color='black')
    ax1.text(median, 1.0, 'IQR', ha='center', va='center', color='black')
    ax1.text(q3, 0.6, 'Q3', ha='center', va='center', color='black')
    ax1.text(median, -0.6, 'Median', ha='center', va='center', color='black')

    # Whiskers
    ax1.text(median - 1.5*inter_qr, 0.4, 'Q1 - 1.5*IQR', ha='center', va='center',
    ax1.text(q3 + 1.5*inter_qr, 0.4, 'Q3 + 1.5*IQR', ha='center', va='center', colo
    ax1.set_ylim(-1, 1.5)
    ax1.set(xlabel=None)
```

```
In [31]: # Function to creates the plots
def CreatesPlots(feats, col, xys):
    sns.kdeplot(naflld_median[feats], ax=ax2, color='black')
    kde_line = ax2.lines[0]
    xys['xs'] = kde_line.get_xdata()
    xys['ys'] = kde_line.get_ydata()

    #..... I wrote this code
    # Creates the box plot with the 25, 50, 75 quartiles
    sns.boxplot(x=naflld_median[feats], color=col, whis=1.5, ax=ax1)

    # Creates a histogram of the age
    ax2.hist(naflld_median[feats], bins=20, color=col, edgecolor='#404040', density=T
    #..... end of code I wrote
```

```
In [32]: # function to add labels to the histogram
def HistLabels(feats):
    plt.xlabel(feats.capitalize())
    plt.ylabel('Density')
    plt.axvline(stats_ct_DF.loc[feats].loc['mean'], linestyle=':', c='r', label='mean')
    plt.axvline(stats_ct_DF.loc[feats].loc['median'], linestyle=':', c='yellow', label='median')
    plt.axvline(stats_ct_DF.loc[feats].loc['mode'], linestyle=':', c='black', label='mode')
    plt.legend(loc='best', facecolor='lightgray')
```

```
In [33]: def CalcPercentage(data_name, q1, q3, iqt):
    pge_qr = {'L.outlier': None, '25': None, 'IQR': None, '75': None, 'R.outlier': None}
    d_ser = nafld_median[data_name]

    ser_len = len(d_ser)
    l_outl = len(d_ser.loc[lambd cell : (cell < q1 - 1.5 * iqt)])
    q1_len = len(d_ser.loc[lambd cell : (cell >= q1 - 1.5 * iqt) & (cell < q1)])
    mid_len = len(d_ser.loc[lambd cell : (cell >= q1) & (cell < q3)])
    q3_len = len(d_ser.loc[lambd cell : (cell >= q3) & (cell < q3 + 1.5 * iqt)])
    r_outl = len(d_ser.loc[lambd cell : (cell >= q3 + 1.5 * iqt)])

    pge_qr['L.outlier'] = l_outl / len(d_ser) * 100
    pge_qr['25'] = q1_len / len(d_ser) * 100
    pge_qr['IQR'] = mid_len / len(d_ser) * 100
    pge_qr['75'] = q3_len / len(d_ser) * 100
    pge_qr['R.outlier'] = r_outl / len(d_ser) * 100

    return pge_qr
```

```
In [34]: # Creates subplots for the age data distribution
fig, (ax1, ax2) = plt.subplots(nrows=2, sharex=True)
fig.suptitle('Histogram of the Age')
fig.set_figwidth(10)

# calculate quartiles
q1, median, q3 = np.percentile(nafld_median['age'], [25, 50, 75])
inter_qr = q3 - q1
pge_qr = CalcPercentage('age', q1, q3, inter_qr)

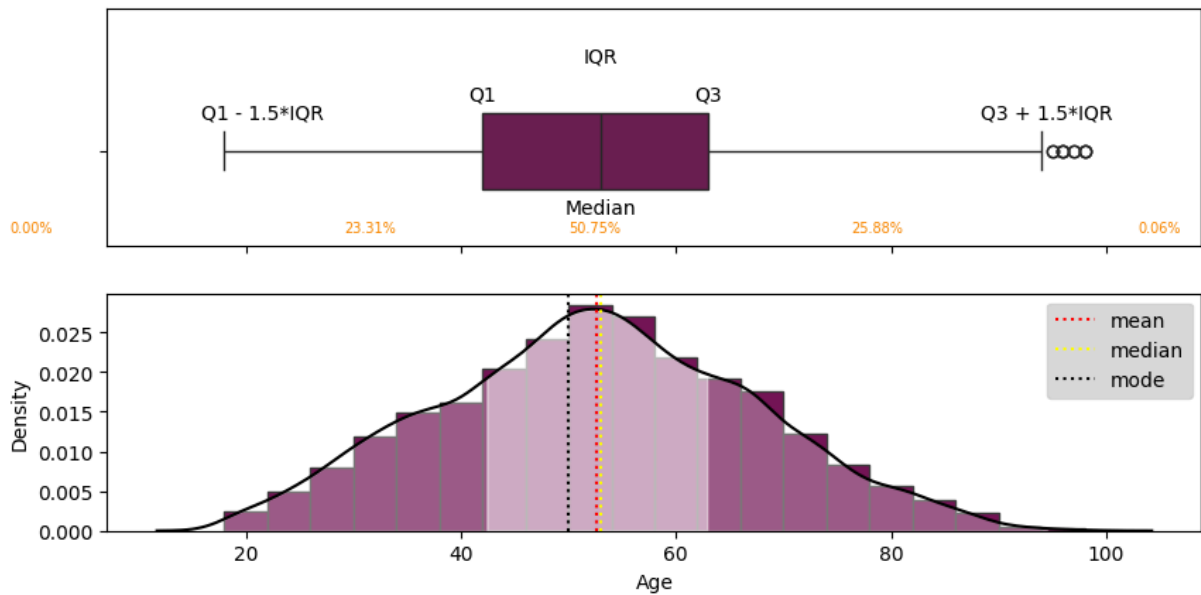
#calls the function to create the plots
xys = {'xs':None, 'xy':None}
CreatesPlots('age', '#761457', xys)

# Call function to format the histogram
HistLabels('age')

# Call function to format the plots
FormatPlots(ax1, ax2, xys['xs'], xys['ys'], q1, q3, inter_qr, median, pge_qr)

plt.show()
```

Histogram of the Age



- **Weight Visualisation**

```
In [35]: # Creates subplots for the age data distribution
fig, (ax1, ax2) = plt.subplots(nrows=2, sharex=True)
fig.suptitle('Histogram of the Weight')
fig.set_figwidth(10)

# calculate quartiles
q1, median, q3 = np.percentile(nafld_median['weight'], [25, 50, 75])
inter_qr = q3 - q1
pge_qr = CalcPercentage('weight', q1, q3, inter_qr)

#calls the function to create the plots
xys = {'xs':None, 'xy':None}
CreatesPlots('weight', '#145C76', xys)

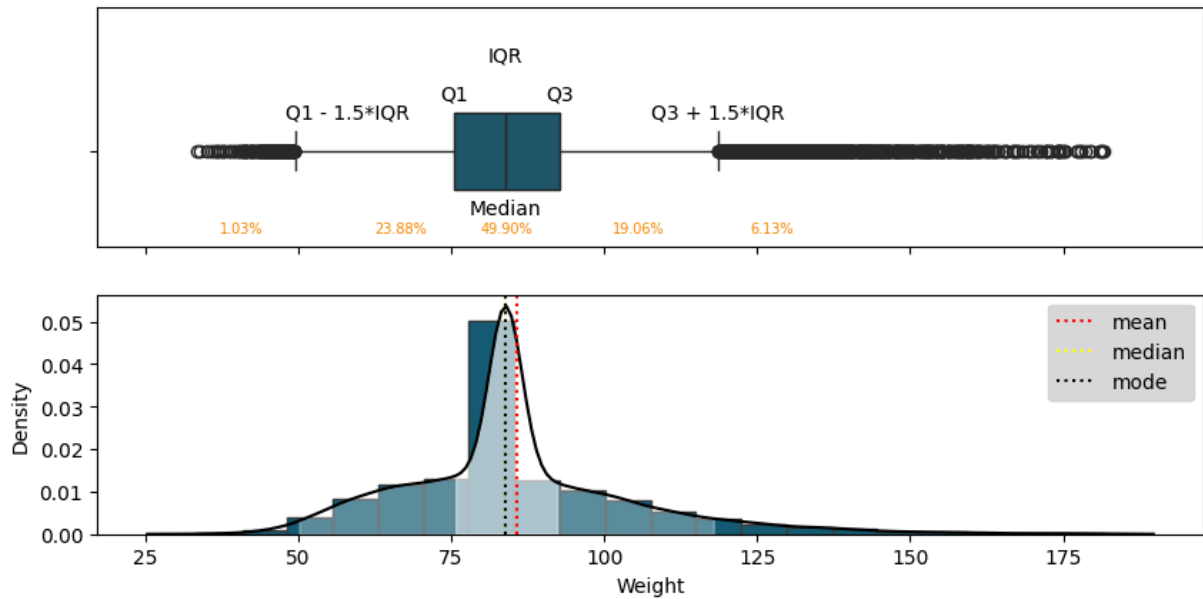
# Call function to format the histogram
HistLabels('weight')

# Call function to format the plots
FormatPlots(ax1, ax2, xys['xs'], xys['ys'], q1, q3, inter_qr, median, pge_qr)

plt.show
```

Out[35]: <function matplotlib.pyplot.show(close=None, block=None)>

Histogram of the Weight



- Height Visualisation

```
In [36]: # Creates subplots for the age data distribution
fig, (ax1, ax2) = plt.subplots(nrows=2, sharex=True)
fig.suptitle('Histogram of the Height')
fig.set_figwidth(10)

# calculate quartiles
q1, median, q3 = np.percentile(nafld_median['height'], [25, 50, 75])
inter_qr = q3 - q1
pge_qr = CalcPercentage('height', q1, q3, inter_qr)

#calls the function to create the plots
xys = {'xs':None, 'xy':None}
CreatesPlots('height', '#6E8500', xys)

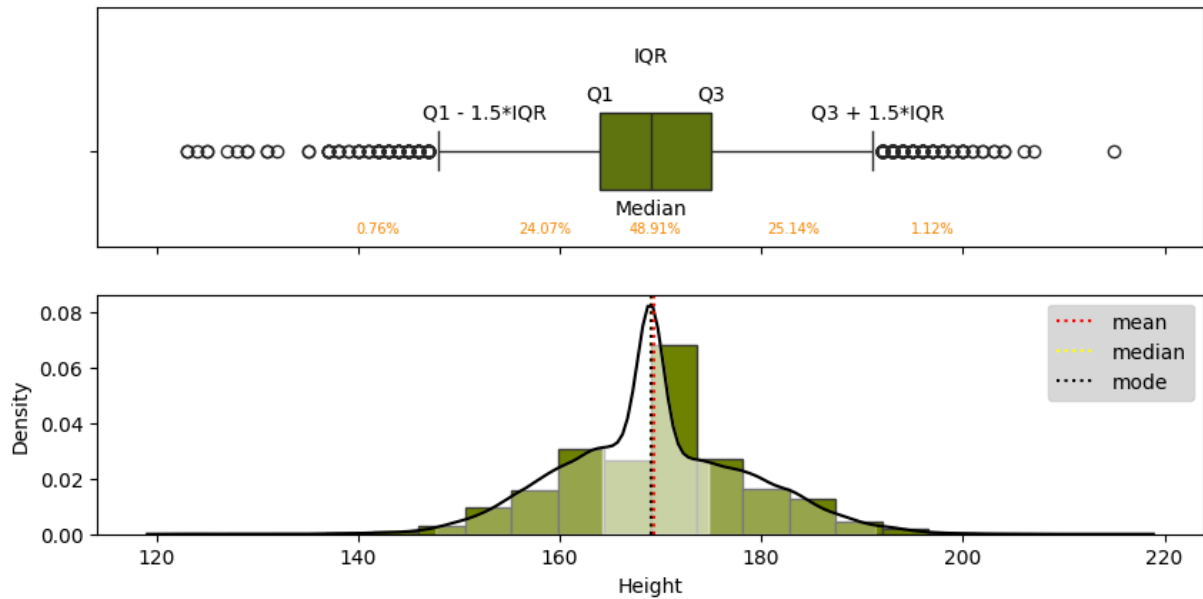
# Call function to format the histogram
HistLabels('height')

# Call function to format the plots
FormatPlots(ax1, ax2, xys['xs'], xys['ys'], q1, q3, inter_qr, median, pge_qr)

plt.show
```

Out[36]: <function matplotlib.pyplot.show(close=None, block=None)>

Histogram of the Height



- **BMI Visualisation**

```
In [37]: # Creates subplots for the age data distribution
fig, (ax1, ax2) = plt.subplots(nrows=2, sharex=True)
fig.suptitle('Histogram of the BMI')
fig.set_figwidth(10)

# calculate quartiles
q1, median, q3 = np.percentile(nafld_median['bmi'], [25, 50, 75])
inter_qr = q3 - q1
pge_qr = CalcPercentage('bmi', q1, q3, inter_qr)

#calls the function to create the plots
xys = {'xs':None, 'xy':None}
CreatesPlots('bmi', '#764A14', xys)

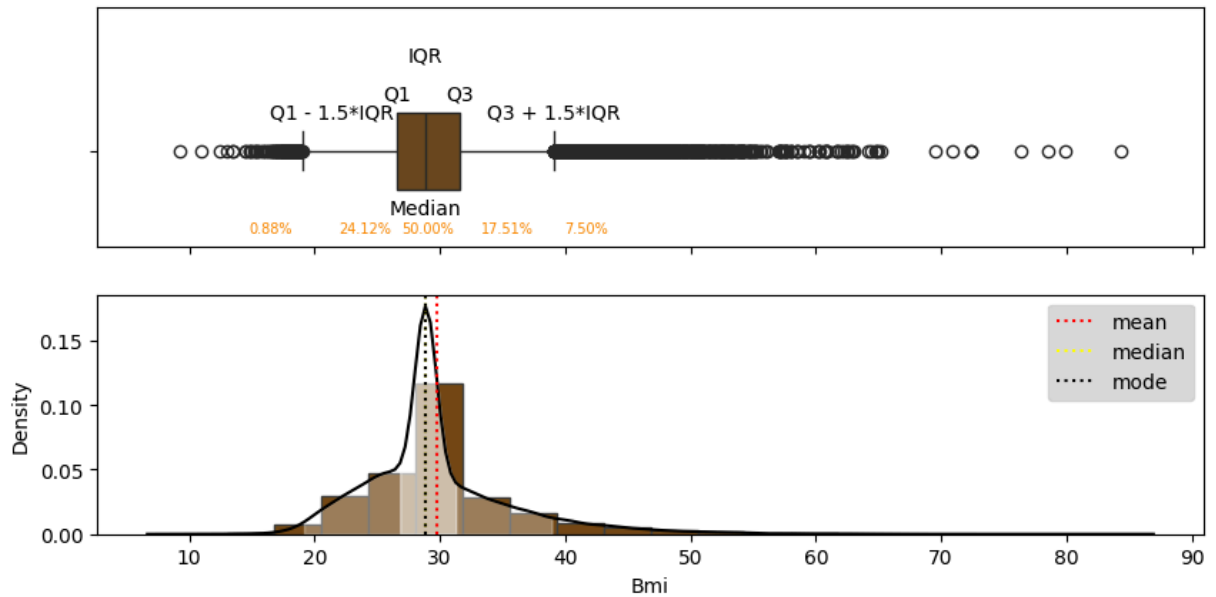
# Call function to format the histogram
HistLabels('bmi')

# Call function to format the plots
FormatPlots(ax1, ax2, xys['xs'], xys['ys'], q1, q3, inter_qr, median, pge_qr)

plt.show
```

Out[37]: <function matplotlib.pyplot.show(close=None, block=None)>

Histogram of the BMI



Bivariate Analysis

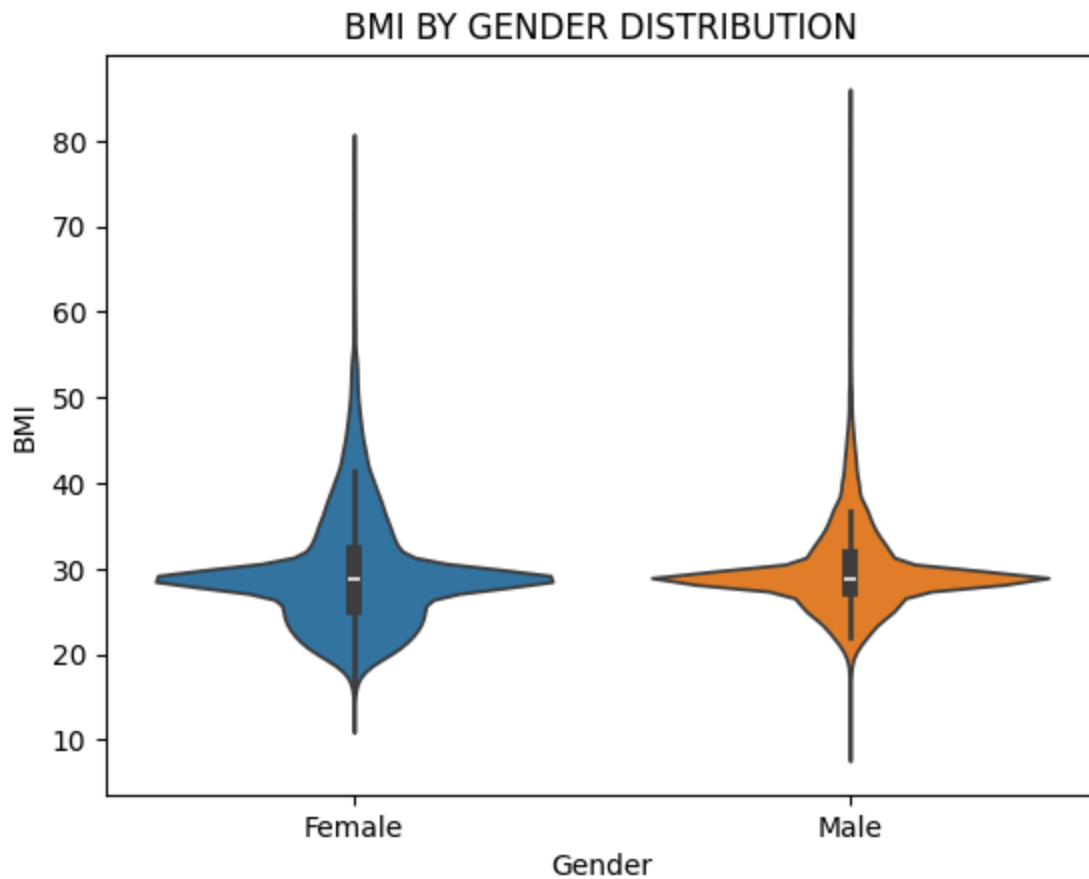
Bivariate analysis combines two or more variables to be displayed in a graph and analyzed.

- **Gender Distribution**

The following visualizations divide the gender feature to combine with the BMI, Weight, Height.

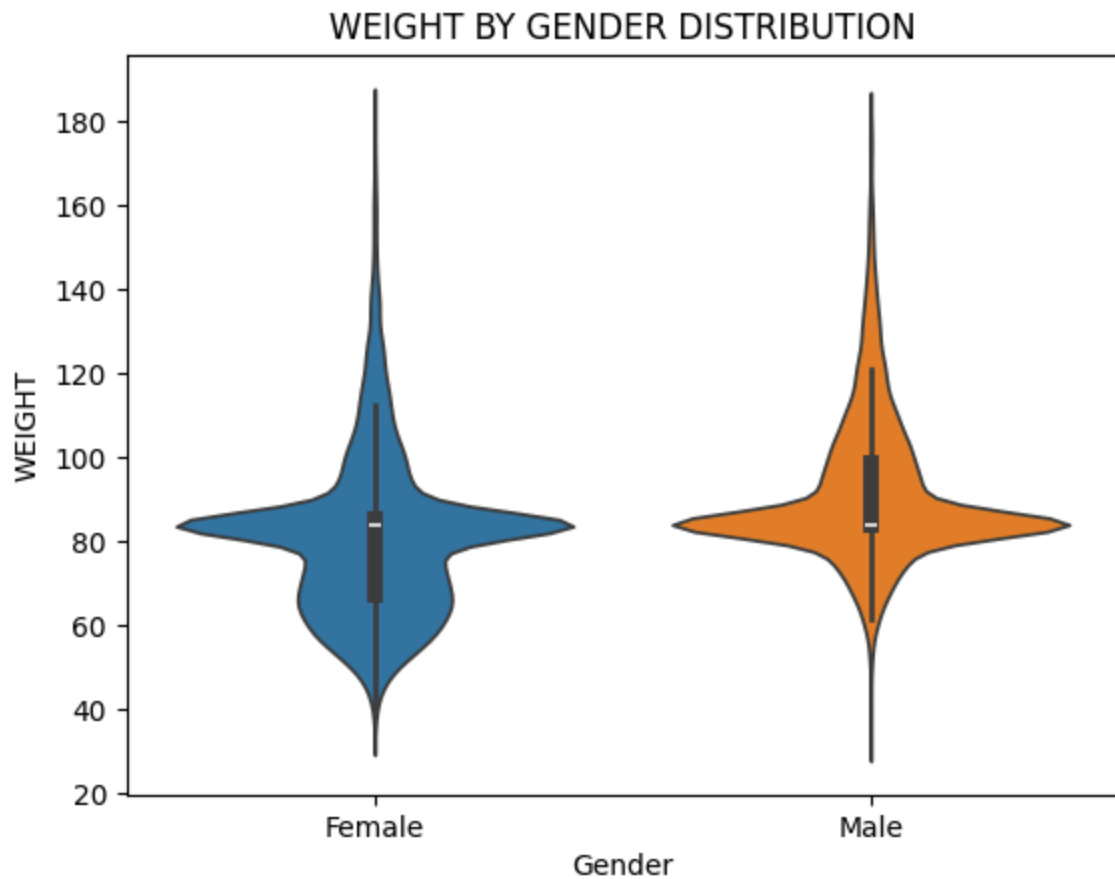
```
In [38]: # Function to display the distribution by gender
def PlotByBivar(x_lbl, bi_arr, feat_name, feat_new_name, graphT):
    box_plot = graphT(x=feat_name, y=x_lbl, hue=feat_name, data=nafl_d_median, legend=True)
    box_plot.set_xticks(range(len(bi_arr)))
    box_plot.set_xticklabels(bi_arr)
    plt.xlabel(feat_new_name)
    plt.ylabel(x_lbl.upper())
    plt.title((x_lbl + ' by ' + feat_new_name + ' Distribution').upper())
    plt.show()
```

```
In [39]: gender = ['Female', 'Male']
PlotByBivar('bmi', gender, 'male', 'Gender', sns.violinplot)
```



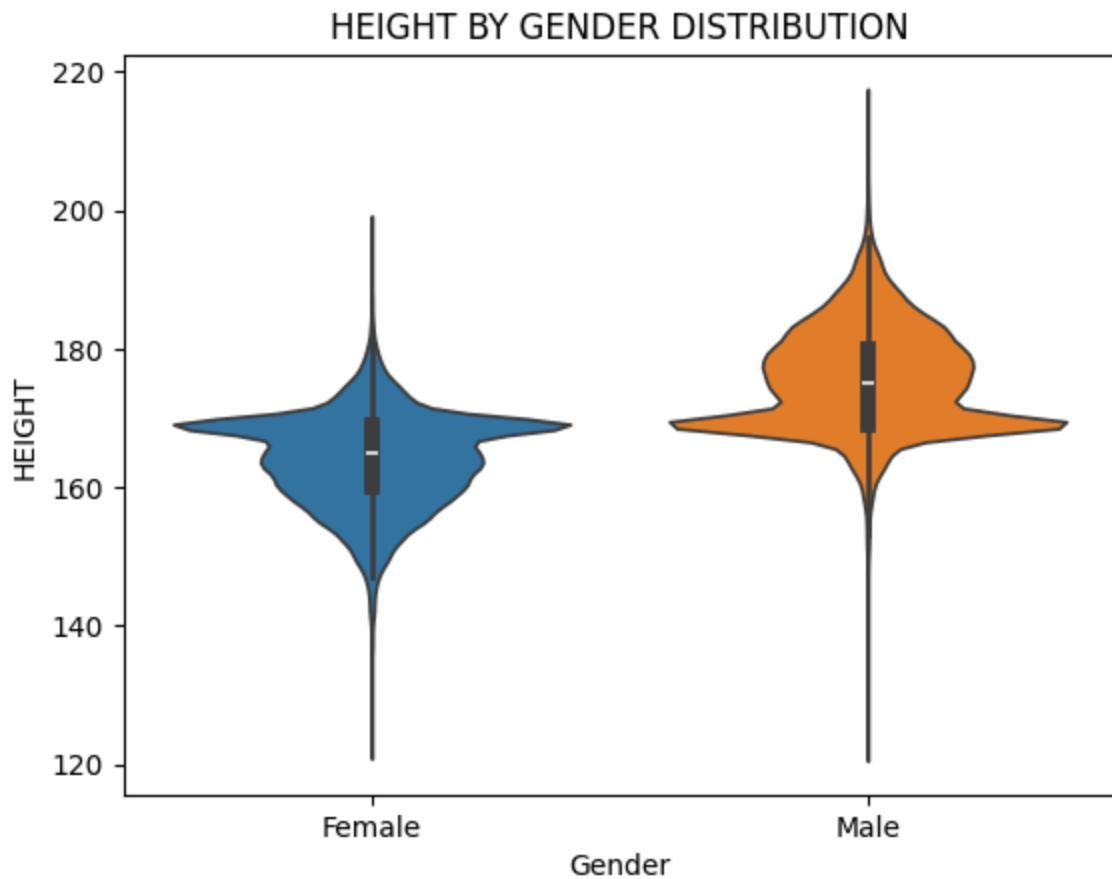
As we can see, in the comparison of the BMI for genders, females and males have the same median. However the concentration of samples around the median is narrower for males. But the outliers are further expanded. This means that the representation is better distributed for woman than men.

```
In [40]: PlotByBivar('weight', gender, 'male', 'Gender', sns.violinplot)
```



The sample weight representation is more inclined to higher weights on males than in females. For females most of the weight samples are between 70 kg and 82 kg, while for men most of the weights are between 80 kg and 100 kg.

```
In [41]: PlotByBivar('height', gender, 'male', 'Gender', sns.violinplot)
```

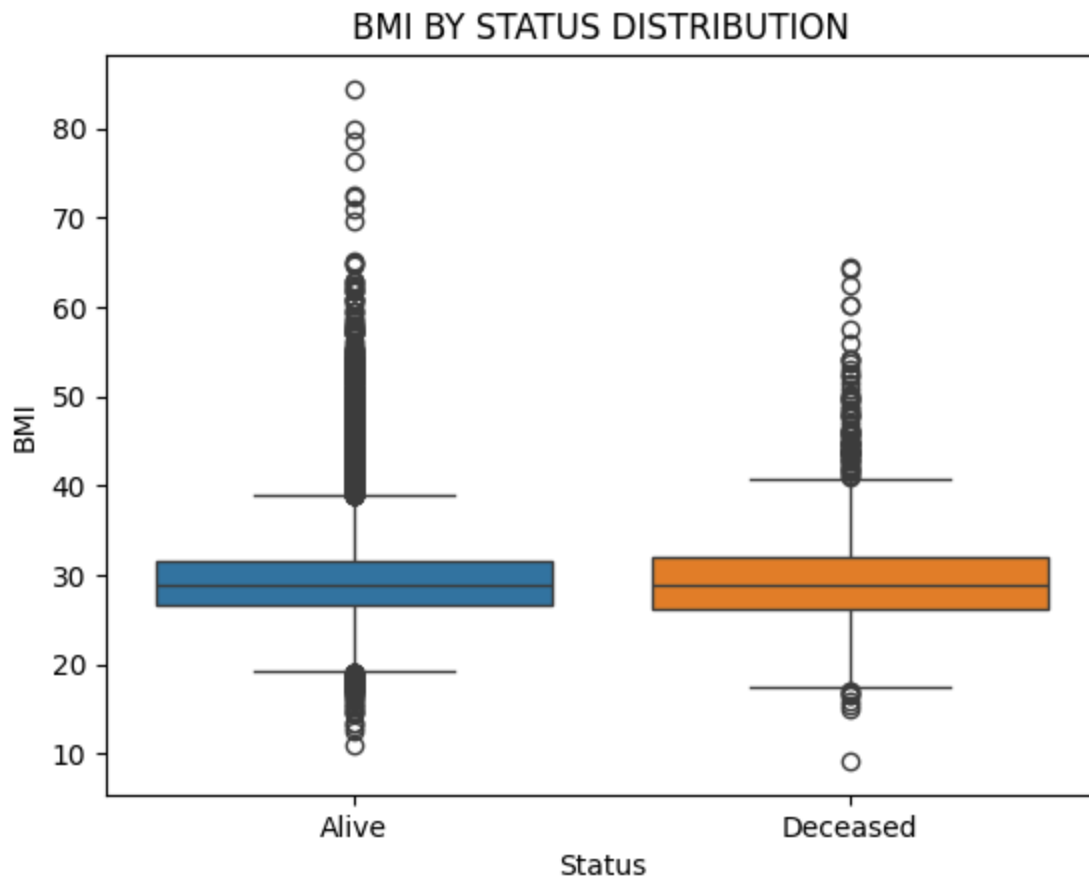


The height distribution on the samples shows that most of the samples are from male between 170 cm and 180 cm, while for females most of the height samples are between 160 cm and 170 cm

- **Vital Status Distribution**

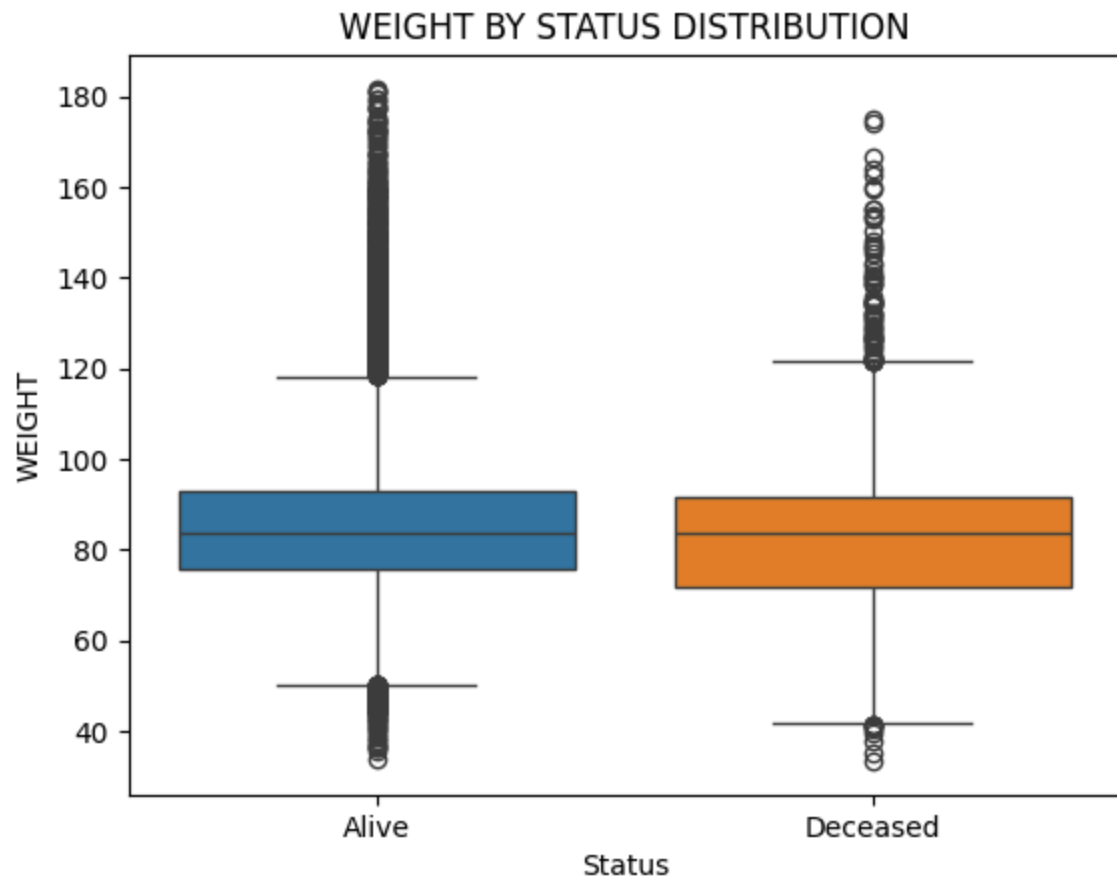
The following visualizations divide the status feature to combine with the BMI, Weight, Height.

```
In [42]: status = ['Alive', 'Deceased']  
PlotByBivar('bmi', status, 'status', 'Status', sns.boxplot)
```

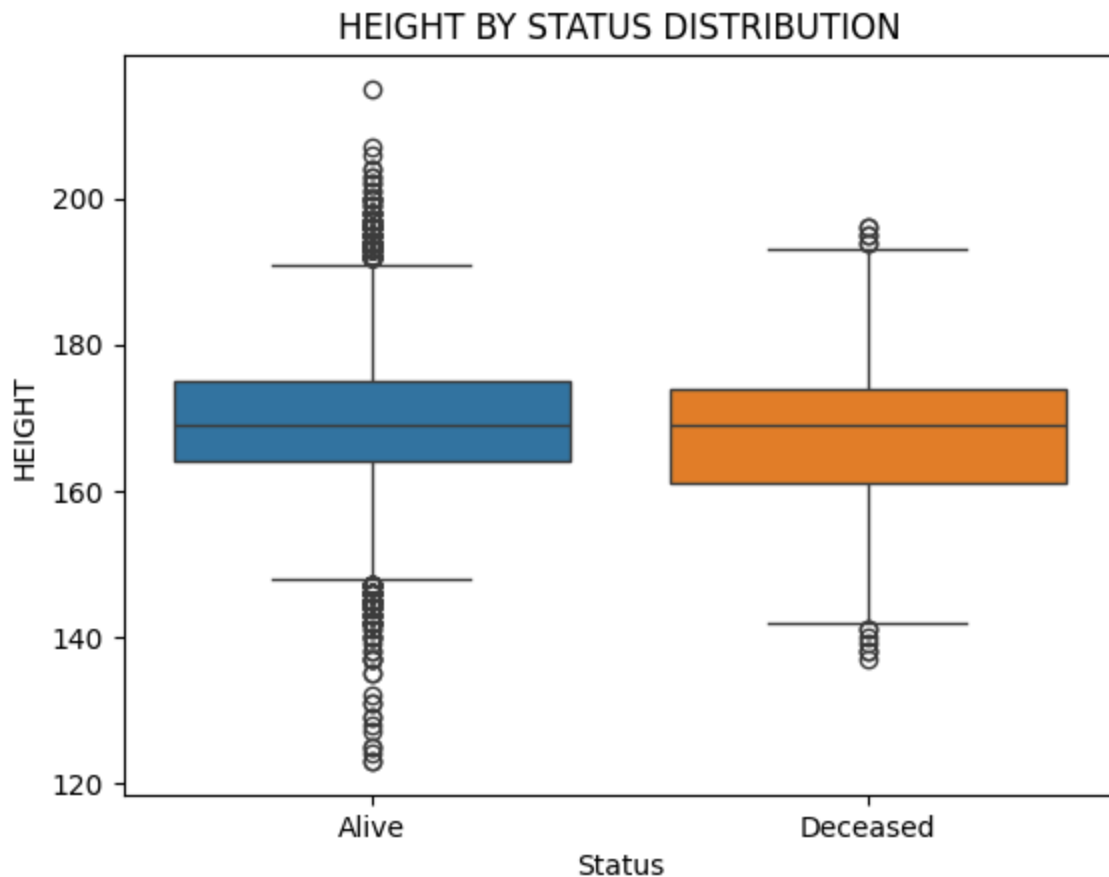


From the status visualization, we can conclude that the people that are alive or deceased have almost the same BMI. Only a very small difference: the deceased group expands a little bit more than the people that are alive, however the median is the same.

```
In [43]: PlotByBivar('weight', status, 'status', 'Status', sns.boxplot)
```



```
In [44]: PlotByBivar('height', status, 'status', 'Status', sns.boxplot)
```



From the previous two plots we can see that the number of people that are deceased have smaller weight or height. However, the difference is too small to make a conclusion. This is because the outliers for the alive subjects are located too far from the mean and there are too many of those.

Feature Engineering Second Iteration

[Table of Contents Link](#)

During the analysis comparisons we can see that the three main variables have too many outliers that can tilt the results of the prediction. Specially the BMI, its outliers are located too far from the mean. Therefore, it is important to find ways to minimize the impact on the analysis.

First, we can understand if there are values that may not be considered real in each feature

```
In [45]: means = [nafld_median[i].mean() for i in nafld_median.columns]
stds = [nafld_median[i].std() for i in nafld_median.columns]
std_l_lim = [i - j * 3 for i, j in zip(means, stds)]
std_r_lim = [i + j * 3 for i, j in zip(means, stds)]

iqrs = [nafld_median[i].quantile(.75) - nafld_median[i].quantile(.25) for i in nafld_median.columns]
iqr_space = [i * 1.5 for i in iqrs]
```

```

iqr_l_lim = [nafld_median[i].quantile(.25) - j for i, j in zip(nafld_median.columns, iqr_space)]
iqr_r_lim = [nafld_median[i].quantile(.75) + j for i, j in zip(nafld_median.columns, iqr_space)]

mins = [nafld_median[i].min() for i in nafld_median.columns]
maxs = [nafld_median[i].max() for i in nafld_median.columns]

stats_table = pd.DataFrame(zip(means, stds, std_l_lim, std_r_lim, iqrs, iqr_space,
                               index=nafld_median.columns,
                               columns=['mean', 'std', 'mean-std(3)', 'mean+std(3)', 'iqr', 'iqr(1.5)']),
                           columns=['mean', 'std', 'mean-std(3)', 'mean+std(3)', 'iqr', 'iqr(1.5)', 'Q1 - iqr(1.5)'])
stats_table

```

Out[45]:

	mean	std	mean-std(3)	mean+std(3)	iqr	iqr(1.5)	Q1 - iqr(1.5)
age	52.659639	14.722515	8.492093	96.827184	21.000000	31.500000	21.000000
male	0.467320	0.498945	-1.029515	1.964155	1.000000	1.500000	-1.000000
weight	85.684267	18.997187	28.692705	142.675829	17.300000	25.950000	58.100000
height	169.356431	9.181985	141.810477	196.902384	11.000000	16.500000	153.000000
bmi	29.735203	6.025065	11.660007	47.810399	5.032355	7.548532	21.510236
status	0.077725	0.267746	-0.725514	0.880964	0.000000	0.000000	0.000000

From the previous chart we can see that the values for the main features are not extreme or impossible to occur in real life, however they are too few cases to accurately represent the group at that level.

In the following two sections we are going to analyze what case is better suited to remove the cases with outliers. As an experiment, the rows are removed using the following two cases:

1. The quartiles and the interquartile multiplied by 1.5
2. The three standard deviations

• Quartiles Method

```

In [46]: # Function to remove rows using the quartiles method
def RemoveRowsByQR(dta, stat_tab, feat):
    new_tab = []
    for i, val in enumerate(dta[feat]):
        if (val > stat_tab['Q1 - iqr(1.5)'][feat] and val < stat_tab['Q3 + iqr(1.5)'][feat]):
            new_tab.append(dta.iloc[i])
    new_tab = pd.DataFrame(new_tab)
    return new_tab

```

```

In [47]: print('Quartiles Method:')
print(' The length of the original table is: ', len(nafld_median))

```

```
print(' The length of the new table is: ', len(RemoveRowsByQR(nafl_d_median, stats_
```

Quartiles Method:

The length of the original table is: 17549

The length of the new table is: 14784

- **Standard Deviation Method**

```
In [48]: # Function to remove rows using the standard deviation method
def RemoveRowsBySTD(dta, stat_tab, feat):
    new_tab = []
    for i, val in enumerate(dta[feat]):
        if (val > stat_tab['mean-std(3)'][feat] and val < stat_tab['mean+std(3)'][f
            new_tab.append(dta.iloc[i])
    new_tab = pd.DataFrame(new_tab)
    return new_tab
```

```
In [49]: print('Standar Deviation Method:')
print(' The length of the original table is: ', len(nafl_d_median))
print(' The length of the new table is: ', len(RemoveRowsBySTD(nafl_d_median, stats_
```

Standar Deviation Method:

The length of the original table is: 17549

The length of the new table is: 17313

From this experiment, we can see that the rows removed by using the standard deviation are fewer cases than the ones removed by the quartiles method, therefore it is more appropriated to remove only the most extrem cases.

- **Removing the rows by Standard Deviation**

```
In [50]: # Remove rows with extrem values in the weight column
print('Remove from Weight Feature: ')
nafl_d_clean = RemoveRowsBySTD(nafl_d_median, stats_table, 'weight')
print(' The length of the original table is: ', len(nafl_d_median))
print(' The length of the new table is: ', len(nafl_d_clean))

# Remove rows with extrem values in the height column
print('Remove from Weight Feature: ')
nafl_d_clean = RemoveRowsBySTD(nafl_d_clean, stats_table, 'height')
print(' The length of the original table is: ', len(nafl_d_clean))
print(' The length of the new table is: ', len(nafl_d_clean))

# Remove rows with extrem values in the bmi column
print('Remove from Weight Feature: ')
nafl_d_clean = RemoveRowsBySTD(nafl_d_clean, stats_table, 'bmi')
print(' The length of the original table is: ', len(nafl_d_clean))
print(' The length of the new table is: ', len(nafl_d_clean))
print( )
```

```
print('The TOTAL of rows removed with extremes outliers are: ', len(nafl_d_median) -  
print('The percentage of remaining data is: ', '%.3f' % (len(nafl_d_clean) / len(naf
```

Remove from Weight Feature:

The length of the original table is: 17549

The length of the new table is: 17313

Remove from Weight Feature:

The length of the original table is: 17250

The length of the new table is: 17250

Remove from Weight Feature:

The length of the original table is: 17115

The length of the new table is: 17115

The TOTAL of rows removed with extremes outliers are: 434

The percentage of remaining data is: 97.527 %

Machine Learning Model

[Table of Contents Link](#)

Linear Regression

In this section, the linear regression model will be implemented to predict the life status of a person diagnosed with non-alcoholic fatty liver disease

- **Class model**

The first model to use is Linear Regression

```
In [51]: # Select class model  
from sklearn.linear_model import LinearRegression
```

- **Initiate model**

```
In [52]: # Initiate the model and assign to variable  
# As a parameter, we add the intercept where the line intercepts the y axis.  
lr_model = LinearRegression(fit_intercept=True)
```

- **Configure data**

In the first step, the data is separated in two groups the feature matrix and the target vector

```
In [53]: nfl_d_clean.head()
```

```
Out[53]:
```

	age	male	weight	height	bmi	status
0	57.0	0.0	60.0	163.0	22.690939	0.0
1	67.0	0.0	70.4	168.0	24.884028	0.0
2	53.0	1.0	105.8	186.0	30.453537	0.0
3	56.0	1.0	109.3	170.0	37.830100	0.0
4	68.0	1.0	83.9	169.0	28.875884	1.0

The feature matrix is made of the columns 'age', 'male', 'weight', 'height' and 'bmi'. The target vector is the column 'status'. The model is train to predict the correlation between the features and the 'status' vector

```
In [54]: # Feature matrix
nafld_feats = nafld_clean.drop('status', axis=1)
# Target vector
nafld_tgt = nafld_clean['status']
```

Divide data set between two groups: the training data and the test data using the train_test_split from sklearn.

```
In [55]: from sklearn.model_selection import train_test_split

#split data set in two groups with test data size of 33%
X_train, X_test, y_train, y_test = train_test_split(nafld_feats, nafld_tgt, test_si
```

```
In [56]: print('Size of train data: ', len(X_train))
print('Size of test data: ', len(X_test))
```

```
Size of train data: 11467
Size of test data: 5648
```

- **Fit the model to data**

```
In [57]: # Fit model
lr_model.fit(X_train, y_train)
```

```
Out[57]:
```

▼ LinearRegression ⓘ ?

► Parameters

```
In [58]: # Query the intercept from the model
lr_model.intercept_
```

```
Out[58]: np.float64(-0.2752514491359442)
```

```
In [59]: # Query the coefficients from the model
lr_model.coef_ #.....(Forth)
```

```
Out[59]: array([ 0.00579405,  0.04249611, -0.00208337,  0.00026134,  0.00538821])
```

- **Apply model to unseen data**

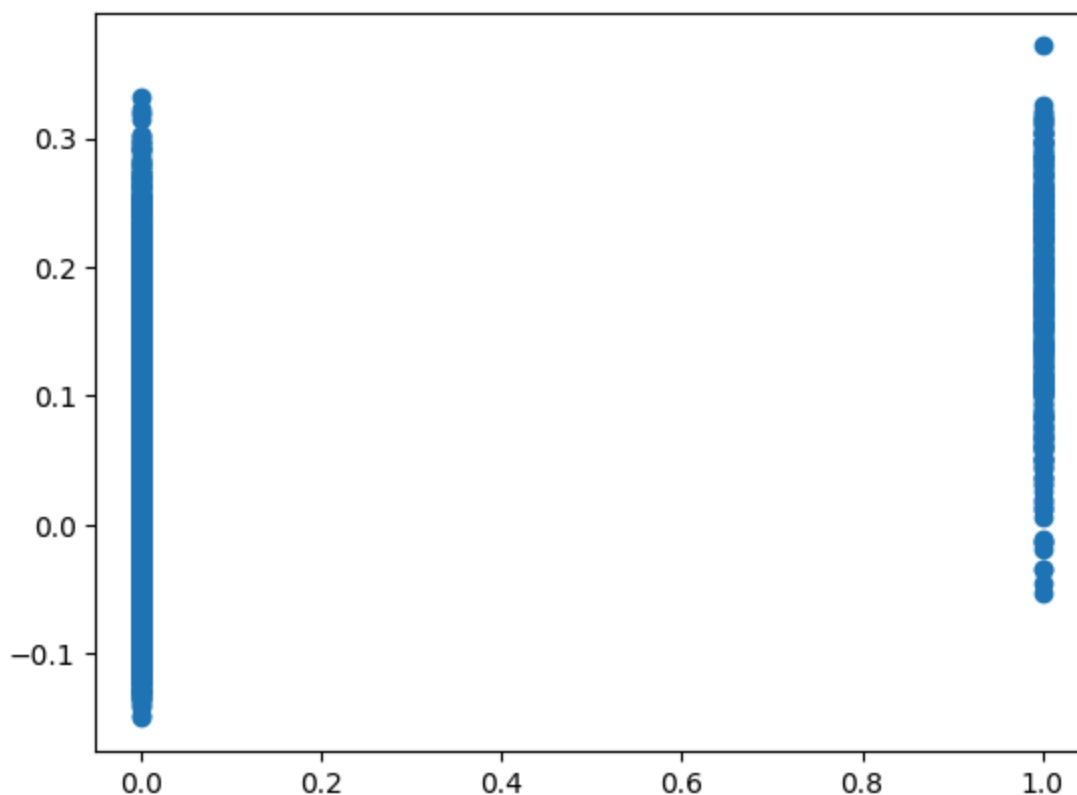
```
In [60]: # predicts a set of a vlues to be compared with the y_test values
y_pred = lr_model.predict(X_test)
y_pred
```

```
Out[60]: array([0.08677791, 0.07689857, 0.06838271, ..., 0.12090848, 0.12595317,
               0.08000432], shape=(5648,))
```

- **Visualize the results**

```
In [61]: plt.scatter(y_test, y_pred)
```

```
Out[61]: <matplotlib.collections.PathCollection at 0x770aee539bd0>
```



During the implementation of this machine learning model: Linear Regression, I found that the Linear Regression model is not appropriate to predict a case with a classification or categorical problems. In this case the target column is made of a boolean (alive or not alive). Every time there is a point added to the chart the regression line adjusts drastically, therefore given an inaccurate result. That is why it is better to find a model that would be better suited for this type of data and would make accurate predictions.

Beyond Linear Regression, a Classification Case for Logistic Regression

Logistic Regression is a machine learning model where the dependent variable is dichotomous (Banoula) meaning dividend in two parts for example binary. "The logistic function is also known as the sigmoid function. The value of this logistic function lies between zero and one"(Banoula). For implementing Logistic Regression, the following cases have to be evaluated:

1. The target variable is binary
2. The binary 1 need to be present to predict the outcome
3. Variables that add value to the outcome
4. The matrix feature should be independent from each other
5. Needs a large data set

- **Class model**

The model to use is Logistic Regression

```
In [62]: # Select class model  
from sklearn.linear_model import LogisticRegression
```

- **Initiate model**

```
In [63]: # Instantiate the model  
logReg_model = LogisticRegression()
```

- **Configure data**

The first step is to choose the features that are independent from each other to create the features matrix. BMI will be dropped due to the dependency to weight and height. The feature 'male' will be dropped because the values are categorical and will skew the data. The following are the features used for the logistics model:

1. Age
2. Weight
3. Height

```
In [64]: # Drops the column that are not needed  
nafld_feats_log = nafld_clean.drop(['bmi', 'male', 'status'], axis=1)  
nafld_feats_log[:3]
```

```
Out[64]:
```

	age	weight	height
0	57.0	60.0	163.0
1	67.0	70.4	168.0
2	53.0	105.8	186.0

Second step is to create the target vector

```
In [65]: #..... I wrote this code
nafld_tgt_log = nafld_clean['status']
nafld_tgt_log[:5]
#..... end of code I wrote
```

```
Out[65]: 0    0.0
         1    0.0
         2    0.0
         3    0.0
         4    1.0
Name: status, dtype: float64
```

Divide data set between two groups: the training data and the test data using the `train_test_split` from sklearn.

```
In [66]: from sklearn.model_selection import train_test_split

#split data set in two groups with test data size of 33%
X_train_log, X_test_log, y_train_log, y_test_log = train_test_split(nafld_feats_log,
```

```
In [67]: print('Size of train data: ', len(X_train_log))
         print('Size of test data: ', len(X_test_log))
```

```
Size of train data: 11467
Size of test data: 5648
```

- **Fit the model to data**

```
In [68]: # Fit model
logReg_model.fit(X_train_log, y_train_log)
```

```
Out[68]:
```

▼ LogisticRegression ⓘ ?

► Parameters

```
In [69]: # Query the intercept from the model
logReg_model.intercept_
```

```
Out[69]: array([-9.18553015])
```

```
In [70]: # Query the coefficients from the model
logReg_model.coef_
```

```
Out[70]: array([[0.10022641, 0.00176879, 0.00266373]])
```

- **Apply model to unseen data**

```
In [71]: # predicts a set of a vlues to be compared with the y_test values
y_pred = logReg_model.predict(X_test_log)
y_pred
```

```
Out[71]: array([0., 0., 0., ..., 0., 0., 0.], shape=(5648,))
```

```
In [72]: # Verify we are receiving positive values
len(y_pred[y_pred > 0])
```

```
Out[72]: 68
```

Model Evaluation

[Table of Contents Link](#)

To assess the performance of the logistic regression model, several evaluation metrics were calculated. While overall accuracy provides a general indication of model performance, additional metrics are particularly important because the dataset is imbalanced. Therefore, precision, recall, F1-score, balanced accuracy, and graphical evaluation methods are included to provide a more comprehensive assessment of the classifier.

Accuracy Scores

Using two different ways we check the model score

```
In [73]: # Accuracy score on training data
print('Training Data Accuracy Score: ', logReg_model.score(X_train_log, y_train_log))
```

```
Training Data Accuracy Score: 0.9280544170227609
```

```
In [74]: # Accuracy score on test data method 1
print('Testing Data Accuracy Score 1: ', logReg_model.score(X_test_log, y_test_log))
```

```
Testing Data Accuracy Score 1: 0.9259915014164306
```

```
In [75]: # Accuracy score on test data method 2
from sklearn.metrics import accuracy_score
# calculate the accurarcy score
print('Testing Data Accuracy Score 1: ', accuracy_score(y_test_log, y_pred))
```

```
Testing Data Accuracy Score 1: 0.9259915014164306
```

The model achieved an overall accuracy of 92%. However, this result should be interpreted with caution because the dataset is highly imbalanced. Since the majority of patients

survived, a classifier can obtain a high accuracy by predicting the majority class most of the time. Therefore, accuracy alone does not provide a complete measure of model performance. Metrics such as precision, recall, F1-score, and the confusion matrix reveal that the model struggles to correctly identify deceased patients, suggesting that further techniques—such as class balancing, threshold tuning, or alternative algorithms—would be needed to improve predictive performance.

Confusion Matrix

The confusion matrix summarizes the number of correct and incorrect predictions made by the classifier. It provides insight into the model's ability to correctly identify both surviving and deceased patients. Confusion matrix to visualize the performance a classification algorithm by showing:

1. True positives
2. False Positives
3. True Negatives
4. False Negatives

```
In [84]: from sklearn.metrics import (
          confusion_matrix,
          ConfusionMatrixDisplay,
          classification_report,
          precision_score,
          recall_score,
          f1_score,
          balanced_accuracy_score,
          roc_auc_score
        )

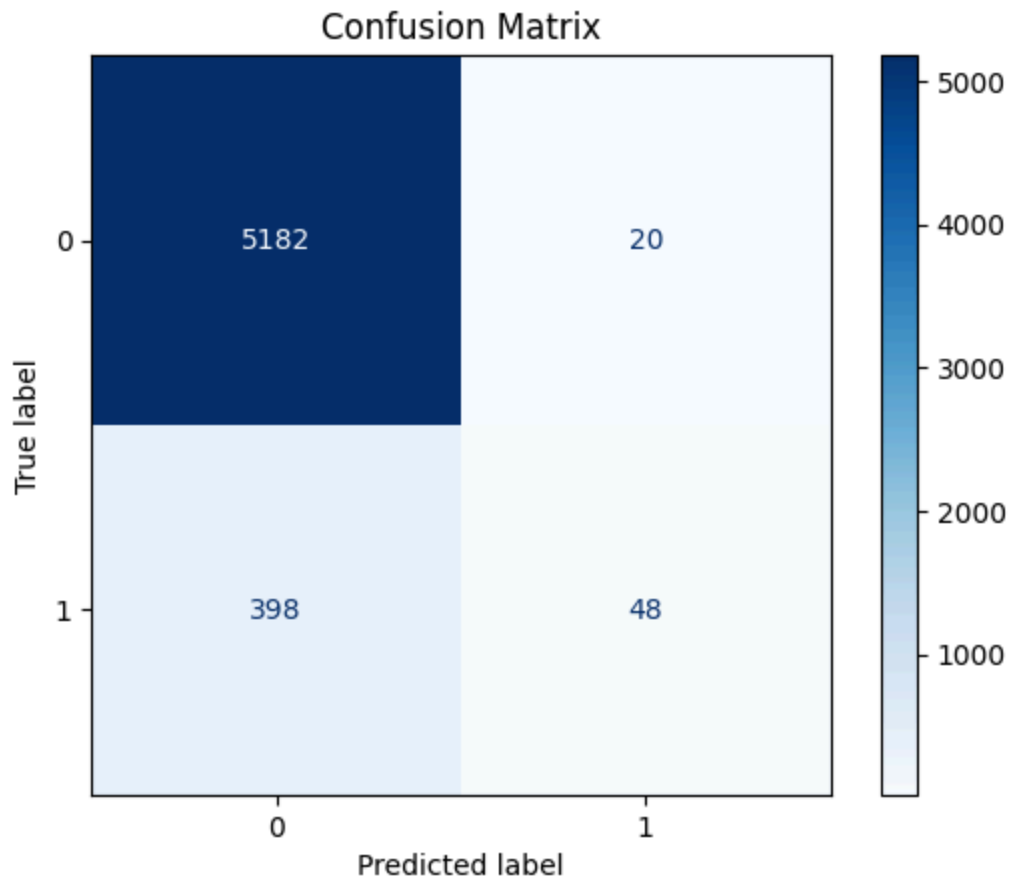
conf = confusion_matrix(y_test_log, y_pred)

TN, FP, FN, TP = conf.ravel()

print(f"True Positive (TP): {TP}")
print(f"False Positive (FP): {FP}")
print(f"True Negative (TN): {TN}")
print(f"False Negative (FN): {FN}")

disp = ConfusionMatrixDisplay(confusion_matrix=conf)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```

```
True Positive (TP): 48
False Positive (FP): 20
True Negative (TN): 5182
False Negative (FN): 398
```



Classification Metrics

The classification report includes precision, recall, and F1-score for each class. These metrics are particularly useful when evaluating imbalanced datasets because they provide more information than overall accuracy alone.

```
In [85]: # Classification report
print(classification_report(y_test_log, y_pred))
```

	precision	recall	f1-score	support
0.0	0.93	1.00	0.96	5202
1.0	0.71	0.11	0.19	446
accuracy			0.93	5648
macro avg	0.82	0.55	0.57	5648
weighted avg	0.91	0.93	0.90	5648

Balanced Accuracy

```
In [86]: print(f"Balanced Accuracy : {balanced_accuracy_score(y_test_log, y_pred):.3f}")
print(f"Precision : {precision_score(y_test_log, y_pred):.3f}")
print(f"Recall : {recall_score(y_test_log, y_pred):.3f}")
print(f"F1 Score : {f1_score(y_test_log, y_pred):.3f}")
```

```
Balanced Accuracy : 0.552
Precision          : 0.706
Recall             : 0.108
F1 Score           : 0.187
```

The model achieved an overall accuracy of 92%. However, this result should be interpreted with caution because the dataset is highly imbalanced. Since the majority of patients survived, a classifier can obtain a high accuracy by predicting the majority class most of the time. Therefore, accuracy alone does not provide a complete measure of model performance. Metrics such as precision, recall, F1-score, and the confusion matrix reveal that the model struggles to correctly identify deceased patients, suggesting that further techniques—such as class balancing, threshold tuning, or alternative algorithms—would be needed to improve predictive performance.

ROC Curve

The Receiver Operating Characteristic (ROC) curve illustrates the trade-off between the true positive rate and the false positive rate across different decision thresholds. The Area Under the Curve (AUC) summarizes the model's overall ability to distinguish between the two classes.

```
In [88]: y_prob = logReg_model.predict_proba(X_test_log)[:,-1]
```

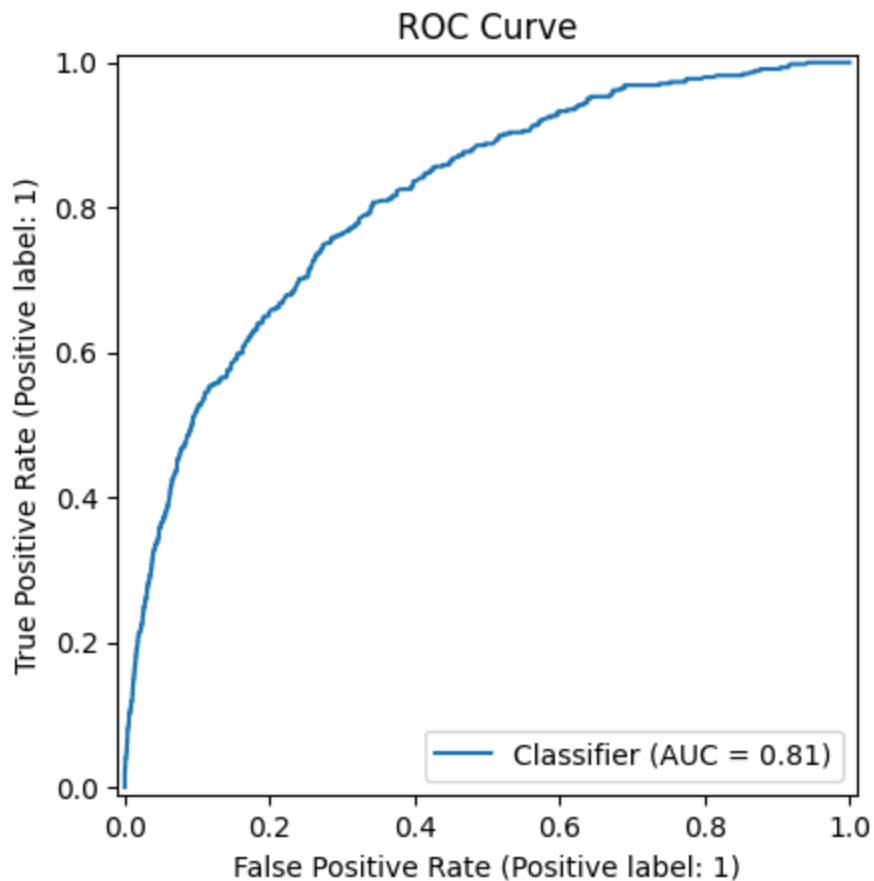
```
/home/mayraosccarr/Projects/nafl-d-prediction/nafl-d-env/lib/python3.10/site-packages/
sklearn/utils/validation.py:2749: UserWarning: X does not have valid feature names,
but LogisticRegression was fitted with feature names
  warnings.warn(
```

```
In [89]: from sklearn.metrics import RocCurveDisplay

RocCurveDisplay.from_predictions(
    y_test_log,
    y_prob
)

plt.title("ROC Curve")
plt.show()

print("ROC AUC:", roc_auc_score(y_test_log, y_prob))
```



ROC AUC: 0.8115607053513396

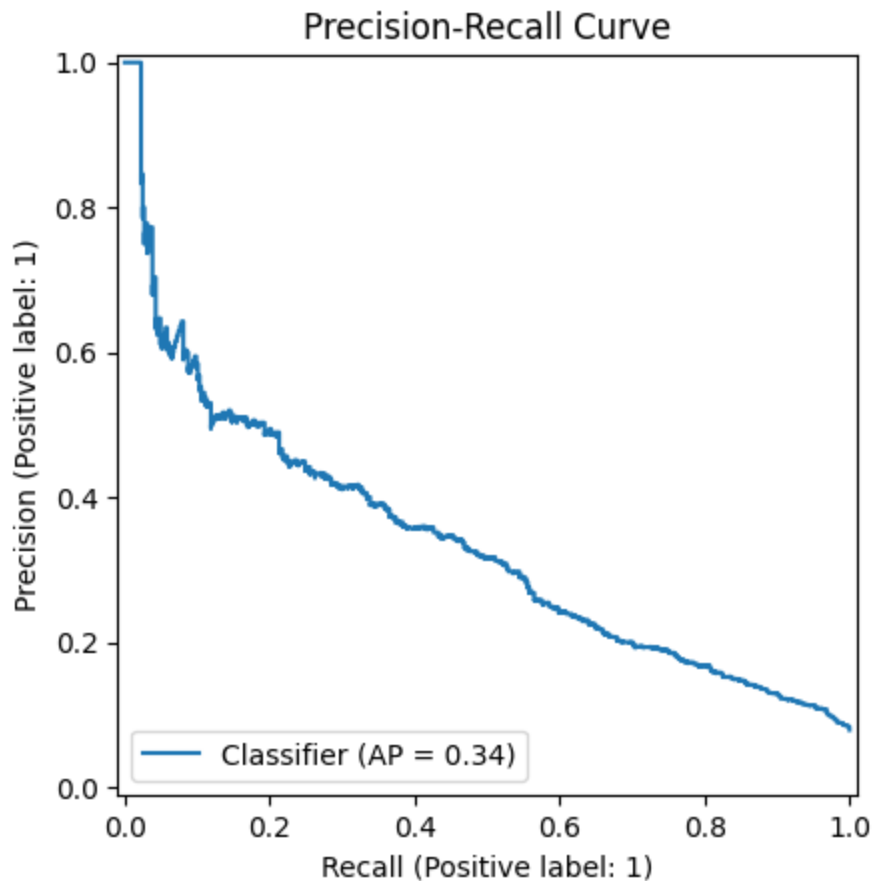
Precision-Recall Curve

Because the dataset contains substantially more surviving than deceased patients, the Precision-Recall curve provides an additional measure of model performance for the minority class. This visualization is especially informative when evaluating imbalanced classification problems.

```
In [90]: from sklearn.metrics import PrecisionRecallDisplay

PrecisionRecallDisplay.from_predictions(
    y_test_log,
    y_prob
)

plt.title("Precision-Recall Curve")
plt.show()
```



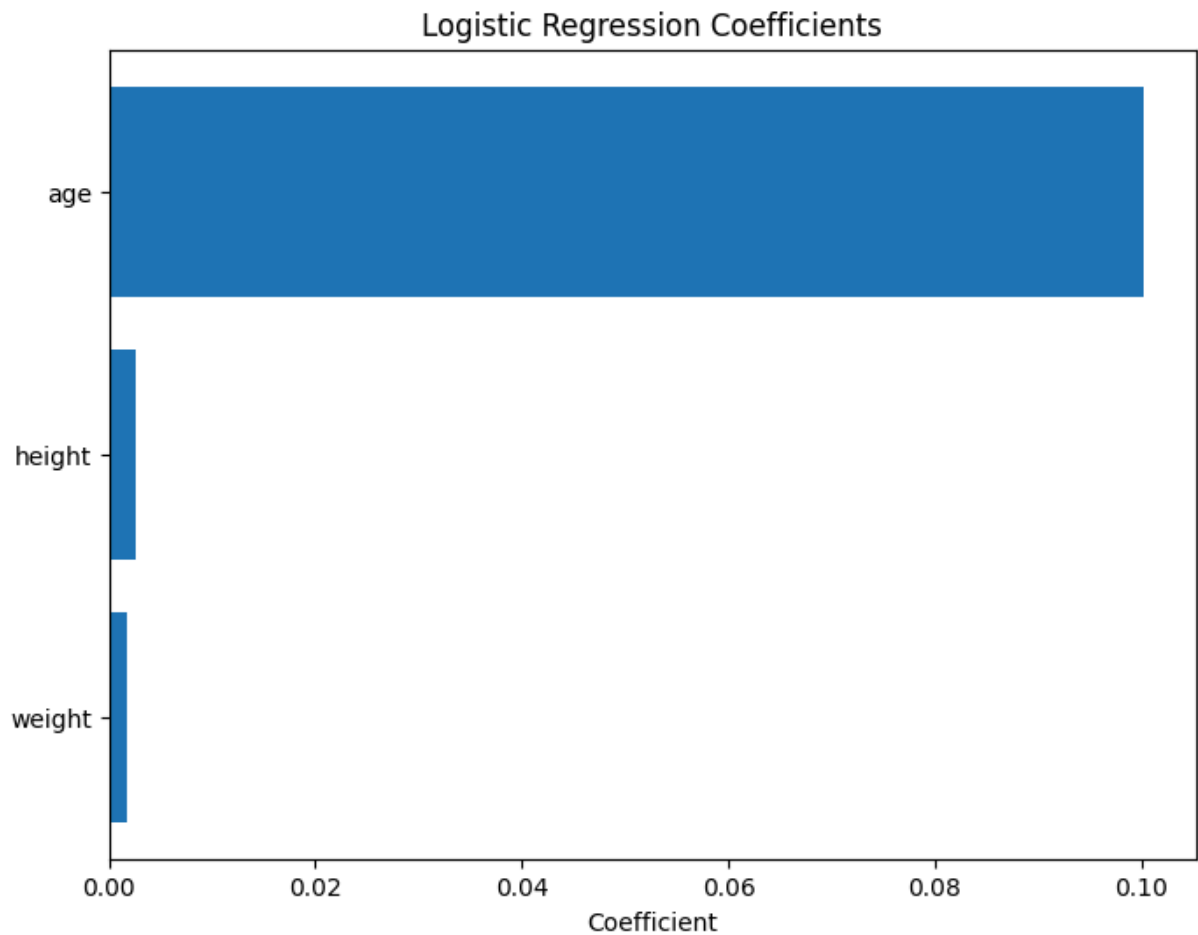
Feature Importance

The logistic regression coefficients indicate the relative contribution of each predictor variable to the model's decision-making process. Positive coefficients increase the likelihood of predicting the deceased class, whereas negative coefficients increase the likelihood of predicting the surviving class.

```
In [92]: coef = pd.DataFrame({
    "Feature": X_train_log.columns,
    "Coefficient": logReg_model.coef_[0]
})

coef = coef.sort_values("Coefficient")

plt.figure(figsize=(8,6))
plt.barh(coef["Feature"], coef["Coefficient"])
plt.title("Logistic Regression Coefficients")
plt.xlabel("Coefficient")
plt.show()
```



- **Odds Ratios**

```
In [93]: coef["Odds Ratio"] = np.exp(coef["Coefficient"])
coef.sort_values("Odds Ratio", ascending=False)
```

```
Out[93]:
```

	Feature	Coefficient	Odds Ratio
0	age	0.100226	1.105421
2	height	0.002664	1.002667
1	weight	0.001769	1.001770

A one-unit increase in this variable multiplies the odds of mortality by this amount.

Results

The logistic regression model achieved an overall accuracy of approximately 92% on the test dataset. At first glance, this suggests strong predictive performance. However, further

evaluation revealed that the dataset is highly imbalanced, with substantially more surviving patients than deceased patients.

The confusion matrix and classification metrics indicate that the model correctly classifies most surviving patients but struggles to identify deceased patients. This is reflected by the relatively low recall for the minority class despite the high overall accuracy.

The ROC and Precision-Recall curves provide additional insight into the classifier's performance beyond accuracy alone. Together, these evaluation methods demonstrate that although the model performs well for the majority class, its ability to detect the minority class is limited.

Conclusion

[Table of Contents Link](#)

This project demonstrates the application of logistic regression to predict patient survival outcomes using clinical data from individuals diagnosed with nonalcoholic fatty liver disease (NAFLD). The analysis included data exploration, preprocessing, model development, and performance evaluation.

Although the model achieved an overall accuracy of approximately 92%, additional evaluation metrics revealed that accuracy alone is not an adequate measure of performance because of the class imbalance present in the dataset. The classifier successfully identified most surviving patients but showed limited ability to detect deceased patients, highlighting the importance of evaluating models with precision, recall, F1-score, balanced accuracy, and graphical performance metrics.

Overall, this project illustrates both the strengths and limitations of logistic regression when applied to imbalanced medical datasets. Future improvements could include applying class-balancing techniques, optimizing the classification threshold, evaluating additional machine learning algorithms, and incorporating feature selection or engineering to improve prediction of the minority class.

Project Reflection

This project was completed as an early machine learning exercise and provided valuable experience in developing and evaluating classification models. Revisiting the project highlighted the importance of selecting appropriate evaluation metrics for imbalanced datasets and reinforced the need to look beyond overall accuracy when assessing model performance. These lessons have influenced the design and evaluation strategies used in my later machine learning and deep learning projects.

References

[Table of Contents Link](#)

Joshi, Krati. "What is Healthcare Data Analytics? Does it Improve Healthcare?" Emeritus, 1 December 2022, <https://emeritus.org/blog/healthcare-data-analytics-career/>. Accessed 5 July 2023.

Robertson, Sally. "What is Hepatology?" News Medical, <https://www.news-medical.net/health/What-is-Hepatology.aspx>. Accessed 5 July 2023.

Cheemerla, S. and Balakrishnan, M. (2021), Global Epidemiology of Chronic Liver Disease. Clinical Liver Disease, 17: 365-370. <https://doi.org/10.1002/cld.1061>

WHO.int. "Obesity and overweight." World Health Organization (WHO), 9 June 2021, <https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight>. Accessed 5 July 2023.

AASLDpubs. "Who We Are." AASLD, <https://www.aasld.org/who-we-are>. Accessed 5 July 2023.

Allen, Alina M.*; Therneau, Terry M.; Larson, Joseph J.; Coward, Alexandra; Somers, Virend K.; Kamath, Patrick S.. Nonalcoholic fatty liver disease incidence and impact on metabolic burden and death: A 20 year-community study. Hepatology 67(5):p 1726-1736, May 2018. | DOI: 10.1002/hep.29546

Singh, Utkarsh. "Non-alcohol fatty liver disease (NAFLD)." Kaggle, <https://www.kaggle.com/datasets/utkarshx27/non-alcohol-fatty-liver-disease>. Accessed 5 July 2023.

Tutorialspoint.com. "First Normal Form (1NF)." Tutorialspoint, <https://www.tutorialspoint.com/sql/first-normal-form.htm>. Accessed 6 July 2023.

GeeksforGeeks. "Python - Normal Distribution in Statistics." GeeksforGeeks, 10 May 2023, <https://www.geeksforgeeks.org/python-normal-distribution-in-statistics/>. Accessed 7 July 2023.

Banoula, Mayank. "An Introduction to Logistic Regression in Python." Simplilearn, 22 February 2023, <https://www.simplilearn.com/tutorials/machine-learning-tutorial/logistic-regression-in-python>. Accessed 14 July 2023.